UNIVERSITÉ HASSAN II DE CASABLANCA
Faculté des Sciences Ben M'Sick
DÉPARTEMENT DE MATHÉMATIQUE ET INFORMATIQUE
MASTER DS&BD



RAPPORT DE PROJET DE FIN DE MODULE :

Python pour la Science des Données

Sous le thème

Prédiction de la réussite académique des étudiants á l'aide de scikit-learn

Réalisé par :

BASSIRY Wijdane	Responsable de groupe
MHADA Hiba	Membre de groupe
MOUJOUD Aya	Membre de groupe
GRANDI Soukaina	Membre de groupe
TACE Mouad	Membre de groupe

Exposé le 25 février 2026 devant les membres de jury :

SAID NOUH	Faculté des sciences Ben M'Sick	Encadrant
TAOUFIK AMZIL	Faculté des sciences Ben M'Sick	Co-Encadrant

Année universitaire : 2025-2026

Remerciement

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

À **Mr SAID NOUH,** notre encadrant exceptionnelle, pour son engagement sans limites. Son expertise, sa générosité et son accompagnement constant ont été d'une aide précieuse. Son exigence de qualité et ses conseils avisés ont guidé nos pas, faisant de ce projet une aventure aussi enrichissante qu'inoubliable.

À **Mr AMZIL Taoufik**, notre coencadrant, notre sincère reconnaissance pour son accompagnement au quotidien, ses conseils éclairés et son soutien indéfectible. Sa bienveillance et son expertise ont été des atouts majeurs à chaque étape de ce projet.

Merci à tous pour leur engagement, leur écoute et leur précieux accompagnement tout au long de ce travail.

Nous exprimons notre profonde reconnaissance au doyen de notre faculté, pour son soutien institutionnel précieux qui a grandement facilité la réalisation de ce projet ambitieux. Son engagement envers le développement académique a créé un cadre propice à notre épanouissement intellectuel.

À **nos chers parents**, nous adressons l'expression de notre amour et de notre gratitude éternelle. Leur soutien indéfectible, Leurs encouragements constants et leur compréhension sans faille ont été notre source de force tout au long de ce parcours exigeant. Sans leur présence rassurante et leurs sacrifices discrets, cette réalisation n'aurait pu voir le jour.

Nous tenons à remercier sincèrement **les membres du jury** pour le temps qu'ils nous ont consacré, leur expertise précieuse et leurs commentaires constructifs qui ont permis d'enrichir considérablement la qualité de notre travail. Leurs suggestions avisées ont contribué à parfaire notre réflexion et notre démarche scientifique. Enfin, notre gratitude s'étend à toutes les personnes qui, de près ou de loin, ont participé à cette aventure intellectuelle. Chaque contribution, chaque conseil et chaque mot d'encouragement ont formé autant de pierres précieuses dans l'édification de ce projet. Ces marques de soutien resteront gravées dans notre mémoire comme autant de témoignages précieux de générosité intellectuelle et humaine.

Dédicaces

Nous dédions notre travail :

À nos parents, qui ont constamment été présents, tant dans les périodes d'incertitude que dans les succès. Je vous remercie pour votre amour, votre appui inconditionnel et tous ces petits sacrifices qui passent souvent inaperçus. Votre confiance en nous, même quand nous doutions de nous-mêmes, a fait toute la différence. Sans vous, ce chemin aurait été bien plus difficile à parcourir.

À nos frères et sœurs, compagnons de route et de toutes les luttes. Votre aide, vos recommandations et votre compagnie quotidienne ont été d'un grand soutien. Vous avez su nous motiver, nous divertir quand c'était nécessaire, et nous rappeler qu'on n'était jamais seuls sur cette voie.

À nos amies, qui ont partagé avec nous les hauts comme les bas de cette aventure, nous vous dédions ce travail avec une profonde reconnaissance. Votre présence, vos encouragements et votre soutien ont été précieux à chaque étape, et ont grandement contribué à notre réussite.

À tous nos professeurs, qui ont su éveiller en nous la curiosité, l'envie d'apprendre et le goût de l'effort, nous adressons notre profonde gratitude, Grâce à votre patience, votre minutie et votre dévouement, vous avez laissé votre empreinte sur notre cheminement et aidé à établir les fondements de notre futur. Je vous remercie de nous avoir transmis bien plus que des savoirs.

A tout ceux dont l'amour, la patience et les encouragements silencieux ont accompagné chaque étape de ce parcours. Qu'ils soient proches par leur présence ou simplement par la pensée, ils demeurent les fondations invisibles de nos réussites.

Résumé

Aujourd'hui, aider les étudiants à réussir leurs études est une priorité pour les universités. Face à la hausse des échecs et des abandons, il est important de pouvoir repérer rapidement ceux qui rencontrent des difficultés, pour mieux les accompagner. C'est dans cet esprit que s'inscrit notre projet : créer un outil de prédiction de la réussite académique, en s'appuyant sur l'apprentissage automatique.

Nous avons travaillé à partir de données réelles concernant les étudiants, leurs résultats, leur profil démographique et leur situation socio-économique. Une première étape a consisté à explorer et préparer ces données : gérer les informations manquantes, encoder certaines variables et normaliser les données, pour garantir leurs qualités et leurs fiabilités.

Ensuite, nous avons testé et comparé plusieurs algorithmes de classification Régression Logistique, Random Forest, AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost à l'aide de Scikit-learn. Pour évaluer leur performance, nous nous sommes appuyés sur différentes mesures comme l'exactitude, la précision, le rappel et le score F1. Cela nous a permis de sélectionner LightGBM le modèle le plus adapté pour prédire la réussite académique atteignant 0.828464 dans l'accuray,0.827966 dans F1 score,0.828464 dans recall et 0.946252 dans ROC-AUC.

Pour rendre l'outil accessible, nous avons développé une interface simple et interactive. Elle permet à un utilisateur, un enseignant ou un conseiller de saisir les informations d'un étudiant et d'obtenir une prédiction immédiate sur son parcours académique.

Les résultats sont encourageants : l'apprentissage automatique se révèle un levier pertinent pour anticiper les difficultés des étudiants. En identifiant les risques tôt, il peut aider à mettre en place un accompagnement ciblé et préventif, et ainsi favoriser la réussite de chacun.

Abstract

Today, helping students succeed in their studies is a priority for universities. With failure and dropout rates on the rise, it is crucial to be able to quickly identify those who are struggling so that they can be better supported. This is the spirit behind our project: to create a tool for predicting academic success using machine learning.

We worked with real data about students their grades, demographic profiles, and socioeconomic status. The first step was to explore and prepare this data: managing missing information, encoding certain variables, and normalizing the data to ensure its quality and reliability.

Next, we tested and compared several classification algorithms Régression Logistic, Random Forest, AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost using Scikit-learn. To evaluate their performance, we used various metrics such as accuracy, precision, recall, and F1 score. This enabled us to select LightGBM as the most suitable model for predicting academic success reaching 0.828464 in accuracy**,** 0.827966 in F1-score**,** 0.828464 in recall**,** and 0.946252 in ROC-AUC**.**

To make the tool accessible, we have developed a simple and interactive interface. It allows a user a teacher or counselor to enter a student's information and obtain an immediate prediction of their academic path.

The results are encouraging: machine learning is proving to be an effective tool for anticipating students' difficulties. By identifying risks early on, it can help to put targeted, preventive support in place, thereby promoting success for everyone.

Table des matières

Remerciement 2
Dédicaces 3
Résumé 4
Abstract 5
LISTE DES FIGURES 8
LISTE DES TABLEAUX 10
LISTE DES ABRÉVIATIONS 11
INTRODUCTION GÉNÉRALE 12
CHAPITRE 1 : Problématique et analyse des besoins 13
1. Introduction 13
2. CAHIER DES CHARGES : Système de Prédiction de la Réussite Académique 13
2.1 Vision de projet 13
2.2 Contexte et besoins 13
2.3 Objectifs du Projet 14
2.4. Analyse des Besoins 14
3. Conclusion 14
CHAPITRE 2 : Etat d'art 15
1.Introduction 15
2.La revue de littérature 15
2.synthèse des résultats 16
3.analyse d'état d'art 18
3.1 Répartition des objectifs de recherche 18
3.2 Répartition des approches utilisées 19
3.3 Fréquence des algorithmes utilisés 19
3.4 Types de jeux de données exploités 20
3.5 Utilisation des métriques d'évaluation 20
3.6 Comparaison des performances moyennes (Accuracy) 21
3.7 Comparaison des performances de tous les modèles 21
3.8 Comparaison des prétraitements selon le jeu de donnée 22
4.Conclusion 22
CHAPITRE 3 : Analyse des données et modélisation prédictive 23
1.Introduction 23

2.Méthodologie 23
2.1 Jeu de donnée 23
2.2 Exploration de Jeu de donnée 25
2.3 Prétraitement de Jeu de donnée 28
2.4 Sélection des features 29
2.5 Les algorithmes d'apprentissage automatique 30
2.6 Expériences et résultats 33
2.7 Synthèse comparative des résultats 41
3. Déploiement de l'interface 42
3.1 Architecture globale de l'interface 42
3.2 Architecture Logicielle de l'application 42
4.Présentation des pages de l'interface 43
4.1 Page d'accueil 43
4.2 Page d'inscription 43
4.3 Page de connexion 44
4.4 Page de prédiction 44
4.5 Page de guide 44
4.6 Page d'historique 45
5.Outils utilisés 45
5.1 éditeurs 45
5.2 Environnements de développement 46
5.Conclusion 47
Conclusion 48
BIBLIOGRAPHIE 49
REFERENCE 50

LISTE DES FIGURES

Figure 1-Répartition des objectifs de recherche....19
Figure 2-Répartition des approches utilisées....19
Figure 3-Fréquence des algorithmes utilisés....20
Figure 4-Types de jeux de données exploités....20
Figure 5-Utilisation des métriques d'évaluation....21
Figure 6-Comparaison des performances moyennes (Accuracy)....21
Figure 7-Comparaison des performances de tous les modèles....22
Figure 8-Comparaison des prétraitements selon le jeu de donnée....22
Figure 9-méthodologie....23
Figure 10-Repartition des étudiants....25
Figure 11-notes des semestres....26
Figure 12-Frais scolarité á jour-Étudiant avec dettes....26
Figure 13-Distribution de l'âge à inscription- Réussite par Genre....27
Figure 14-Impact de la bourse sur le statut final....27
Figure 15-Nombres d'unités validées au 2ème semestre par statut....27
Figure 16-La matrice de corrélation....28
Figure 17-Le taux de chômage et le PIB....28
Figure 18-15 variables plus importantes....30
Figure 19-code d'entrainement....34
Figure 20-code pour matrice de confusion....35
Figure 21-matrice de confusion pour régression logistique....35
Figure 22-matrice de confusion pour random forest....36
Figure 23-matrice de confusion pour AdaBoost....36
Figure 24-matrice de confusion pour Gradient Boosting....36
Figure 25-matrice de confusion pour XGBoost....37
Figure 26-matrice de confusion pour LightGBM....37
Figure 27-CatBoost....38
Figure 28-code de la courbe ROC....38
Figure 29-courbe ROC pour régression logistique....39
Figure 30-courbe ROC pour random forest....39
Figure 31-courbe ROC pour AdaBoost....39
Figure 32-courbe ROC pour Gradient Boosting....40
Figure 33-courbe ROC pour XGBoost....40
Figure 34-courbe ROC pour LIghtGBM....41
Figure 35courbe ROC pour CatBoost....41
Figure 36-comparaison des perfermances des modeles....42
Figure 37-architecture globale de l'interface....42
Figure 38-page d'accueil....43
Figure 39-page d'inscription....43
Figure 40-page de connexion....44
Figure 41-page de prédiction....44
Figure 42-page de guide....45
Figure 43-page d'historique....45
Figure 44-logo python....46
Figure 45-logos de pandas....46
Figure 46-logo de joblib....46

Figure 47-logo de VS Code 46
Figure 48-logo anaconda.....46
Figure 49-logo jupyter.....47
Figure 50-logo google colab 47

LISTE DES TABLEAUX

Tableau 1-Besoins fonctionels 14
Tableau 2-Besoins non fonctionels 14
Tableau 3-synthèse d'état d'art ..... **Error! Bookmark not defined.**
Tableau 4-description de dataset 24
Tableau 5-description des caractéristiques 24
Tableau 6-Hyperparamètres de régression logistique 30
Tableau 7-Hyperparamètres de Random Forest 31
Tableau 8-Hyperparamètres de ADABOOST 31
Tableau 9-Hyperparamètres de Gradient Boosting 31
Tableau 10-Hyperparamètres de XGBoost 32
Tableau 11-Hyperparamètres de LightGBM 32
Tableau 12-Hyperparamètres de CatBoost 33
Tableau 13-Résultats de l'entrainement 34

LISTE DES ABRÉVIATIONS

IA : Intelligence Artificielle

ML : Machine Learning

DL: Deep Learning

EDM: Educational Data Mining

SVM : Support Vector Machine

KNN: k-Nearest Neighbors

RF : Random Forest

DT : Decision Tree

LR : Logistic Regression

SVR : Support Vector Regression

MLP : Multi-Layer Perceptron (Perceptron Multi-Couches)

Bi-LSTM: Bidirectional Long Short-Term Memory

XGBoost: Extreme Gradient Boosting

LGBM: Light Gradient Boosting Machine

SMOTE: Synthetic Minority Over-sampling Technique

PCA : Principal Component Analysis (Analyse en Composantes Principales)

UML: Unified Modeling Language

ROC: Receiver Operating Characteristic

AUC: Area Under Curve

RMSE: Root Mean Squared Error

MAE : Mean Absolute Error

INTRODUCTION GÉNÉRALE

Aujourd'hui, la technologie fait partie intégrante de notre quotidien. Elle a profondément changé notre manière d'apprendre et d'enseigner, en renforçant la connectivité et en automatisant certaines tâches dans le monde de l'éducation[1]. Mais cette révolution numérique a aussi son revers : la gestion des données scolaires est devenue un vrai défi. Avec la masse d'informations générée par le parcours de chaque étudiant, il est presque impossible d'assurer un suivi personnalisé avec les méthodes classiques. Résultat, beaucoup d'élèves se retrouvent fragilisés, au risque de décrocher ou d'échouer[2].

La réussite scolaire, ce n'est pas qu'une question de notes. Elle dépend d'une multitude d'éléments, à la fois personnels et pédagogiques, qui aident un étudiant à mener son cursus à bien. On peut regrouper ces facteurs en deux grandes familles. D'un côté, les éléments internes les résultats académiques bien sûr, mais aussi l'assiduité et la motivation personnelle. De l'autre, les facteurs externes, comme la situation socio-économique[3], l'environnement familial ou les conditions d'apprentissage. Le problème, c'est que si on ne repère pas à temps les signaux d'alerte dans ces deux domaines, on laisse souvent s'installer des difficultés qui auraient pu être évitées avec un coup de pouce adapté[4].

C'est là que l'Intelligence Artificielle et l'Apprentissage Automatique entrent en jeu[5]. Grâce à eux, les systèmes de suivi éducatif peuvent désormais analyser des comportements complexes et des données passées pour identifier les trajectoires à risque. Notre projet s'inscrit dans cette dynamique : nous travaillons à développer un modèle innovant en utilisant la bibliothèque Scikit-learn. L'idée est de comparer plusieurs algorithmes pour aboutir à une solution optimisée, capable de prédire les chances de réussite des étudiants[6].

Nous avons structuré notre travail en quatre chapitres. **Le premier chapitre** pose les bases : il présente les objectifs et le contexte du projet. Dans **le deuxième chapitre** on y définit le cahier des charges, Vient ensuite **le troisième chapitre,** le cœur de l'analyse : on y aborde l'exploration, le prétraitement des données et la sélection des caractéristiques importantes, puis on compare les performances de différents modèles d'apprentissage automatique pour les entraîner, les évaluer et choisir le plus pertinent, en proposant des pistes d'amélioration. Enfin, on se concentre sur le déploiement de l'interface, qui permettra concrètement de prédire la réussite académique des étudiants.

CHAPITRE 1 : Problématique et analyse des besoins

1. Introduction

Le contenu de ce chapitre relève de l'analyse et conception du système et constitue le premier bloc du projet de prédiction de la réussite académique, qui a pour but d'analyser le problème et de cerner les besoins fonctionnels et non fonctionnels du système. Un cahier des charges est tout d'abord conçu dans le but de définir le périmètre et les objectifs du projet, puis une conception UML modélise le fonctionnement de celui-ci, ainsi que les interactions entre ses différents composants. Enfin, la présentation de l'architecture globale de l'application, où le code sera produit au cours de la suite de la phase de développement, est effectuée.

2. CAHIER DES CHARGES : Système de Prédiction de la Réussite Académique

2.1 Vision de projet

Notre projet s'inscrit dans le domaine de l'éducation intelligente et de l'analyse des données académiques. Nous avons pour ambition de créer un système capable de prédire la réussite des étudiants en utilisant des techniques de machine learning et d'analyse de données. La réussite académique est un enjeu clé pour les établissements d'enseignement. L'échec et l'abandon scolaire ont en effet un impact direct sur la performance des institutions et sur le parcours des étudiants. Si l'on pouvait repérer plus tôt les étudiants en difficulté, on pourrait leur proposer un accompagnement mieux adapté. C'est là que l'intelligence artificielle entre en jeu. Grâce à des outils comme la bibliothèque Scikit-learn, nous pouvons analyser différentes données académiques — notes, absences, temps d'étude, participation, etc. — pour identifier les facteurs qui influencent la réussite.

2.2 Contexte et besoins

Dans beaucoup d'établissements, les choix pédagogiques reposent encore trop souvent sur l'expérience personnelle ou sur une analyse manuelle des résultats. Pourtant, ces mêmes institutions possèdent aujourd'hui une véritable mine de données académiques, qui reste largement inexploitée.

Intégrer le Machine Learning dans l'éducation permettrait justement de donner du sens à ces données. On pourrait ainsi :

- **Repérer plus tôt les étudiants qui décrochent,**
- **Mieux cerner ce qui influence vraiment la réussite,**
- **Ajuster en conséquence les méthodes d'accompagnement.**

C'est dans cette optique que s'inscrit notre projet : une approche innovante pour rendre la gestion pédagogique plus fine et plus proactive, grâce à l'analyse prédictive.2.1. Présentation de projet

- **Nom du projet :** SmartPredict Student

- **Sujet :** Développement d'un modèle d'intelligence artificielle capable de prédire si un étudiant va réussir ou échouer à partir de ses données académiques et socio-économiques.
- **Problématique :** Comment aider les universités à identifier les étudiants en difficulté avant l'examen final ?
- **Technologie clé :** Python, Scikit-Learn, Pandas.

2.3 Objectifs du Projet

Objectif principal : Fournir un outil d'aide à la décision pour identifier les étudiants "à risque" afin de leur proposer un accompagnement précoce.

Objectifs techniques :

- Atteindre une précision (accuracy) d'au moins 75-80%.
- Produire un code Python modulaire et documenté.
- Gérer l'intégralité du cycle de vie du projet sur JIRA.

2.4. Analyse des Besoins

Besoins Fonctionnels

Tableau 1-Besoins fonctionels

ID	Module	Description de la fonctionnalité
F01	Données	Charger un jeu de données (dataset) contenant les notes, absences et données sociales.
F02	Analyse	Générer des visualisations (corrélations) pour comprendre les causes de l'échec.
F03	Modélisation	Entraîner une IA capable de classer un étudiant (Réussite ou Échec).
F04	Interface	Formulaire de saisie pour tester de nouveaux profils d'étudiants.
F05	Suivi	Afficher la fiabilité du modèle (Accuracy, Précision).

Besoins Non-Fonctionnels

Tableau 2-Besoins non fonctionels

Catégorie	Exigence
Performance	Temps de réponse de la prédiction < 2 secondes.
Précision	Taux de réussite du modèle (Accuracy) supérieur à 75%.
Agilité	Mise à jour quotidienne du tableau JIRA et respect des cérémonies Scrum.

3. Conclusion

Pour conclure, ce chapitre a posé les bases essentielles du système « SmartPredict Student », grâce à une analyse approfondie des besoins et une conception bien structurée. La rédaction du cahier des charges a permis de définir clairement les objectifs fonctionnels et techniques du projet. Par ailleurs, la modélisation UML a offert une vision d'ensemble du fonctionnement du système et de ses différentes composantes. On peut donc affirmer que cette phase d'analyse et de conception constitue un socle solide pour le développement et la mise en œuvre du modèle de prédiction.

CHAPITRE 2 : Etat d'art

1.Introduction

Dans ce chapitre, nous nous plongeons dans la littérature scientifique consacrée à la prédiction de la réussite académique grâce à l'intelligence artificielle et au Big Data. Nous allons passer en revue les différentes approches qui existent, examiner les modèles d'apprentissage automatique et profond les plus utilisés, et nous intéresser aux jeux de données ainsi qu'aux méthodes de prétraitement typiquement employés dans ce domaine de recherche.

2.La revue de littérature

En 2021, Albreiki, Zaki et Alashwal [7] se sont penchées sur le sujet en publiant une revue systématique consacrée à la prédiction des résultats académiques grâce au Machine Learning. Leur analyse passe en revue les principales approches employées dans le domaine de l'Educational Data Mining, comme Random Forest, SVM, Naïve Bayes, k-NN ou encore les arbres de décision. Il en ressort que les modèles supervisés occupent une place prépondérante, Random Forest et SVM figurant parmi les méthodes les plus efficaces. Les autrices mettent aussi en avant le rôle crucial du prétraitement des données et de la sélection des variables pour affiner la précision des prédictions.

En 2022, Yağcı [8] a étudié la prédiction des résultats académiques de 1 854 étudiants ayant suivi le cours de langue turque-I au semestre d'automne 2019-2020. Pour cela, il a testé et comparé plusieurs algorithmes d'apprentissage automatique, dont Random Forest, SVM, les k-plus proches voisins, la régression logistique et Naïve Bayes. Après avoir nettoyé et préparé les données, les modèles ont affiché des taux de précision compris entre 70 % et 75 %. Random Forest s'est légèrement démarqué, confirmant ainsi l'utilité de l'apprentissage supervisé pour anticiper la réussite des étudiants.

Une recherche publiée en 2023 dans [9] a suggéré l'emploi de modèles de régression basés sur l'apprentissage automatique pour anticiper les performances futures des étudiants à partir de leurs résultats précédents. Un jeu de données provenant de Kaggle, contenant des renseignements académiques sur des étudiants en informatique, a été utilisé par les auteurs. Le prétraitement incluait la prise en charge des valeurs absentes et l'ajustement des données à une échelle standard. Les algorithmes évalués comprennent la régression linéaire, la Régression par Support Vector (SVR), la Régression kNN et la Régression par Forêt Aléatoire. Les données expérimentales suggèrent que la régression par forêt aléatoire offre les meilleures performances, mettant en évidence l'efficacité des techniques d'ensemble pour prédire les performances académiques continues.

Au cours de 2023, les chercheurs de [10] ont examiné l'impact des facteurs comportementaux, sociaux et démographiques sur la réussite scolaire des étudiants. Leur recherche se base sur un jeu de données qui fusionne des informations relatives aux pratiques d'apprentissage, à la participation parentale et au profil individuel des élèves. Suite à la préparation des données, une comparaison a été effectuée entre divers algorithmes de classification, dont SVM, l'Arbre de Décision, la Forêt Aléatoire et les k-Plus Proches Voisins. Les conclusions indiquent que le modèle Random Forest affiche les performances les plus élevées globalement, soulignant l'importance cruciale des habitudes d'étude quotidiennes dans la réussite scolaire.

Ramos et ses collaborateurs [11] ont suggéré en 2024 dans leur étude un modèle prédictif anticipé de la réussite éducative des étudiants dès leur première année universitaire. La recherche a été réalisée sur une vaste base de données comprenant 6 690 entrées et 21 variables d'ordre académique et socio-économique. Suite à un traitement minutieux des données, qui comprend la sélection de caractéristiques, nous avons évalué divers algorithmes, notamment Decision Tree et XGBoost. Les données montrent que XGBoost obtient un AUC de 87,75 % et une précision d'environ 80 %, surpassant les modèles traditionnels, ce qui valide l'efficacité des méthodes de boosting pour ce genre de problématique.

Toujours en 2024, une recherche dans [12] a évalué l'efficacité de divers algorithmes d'apprentissage automatique pour prédire le succès académique à partir de données académiques et démographiques recueillies sur plusieurs semestres. Les modèles considérés dans cette évaluation comprennent la régression logistique, SVM, la forêt aléatoire et les réseaux de neurones artificiels. Les données démontrent que Random Forest et les réseaux de neurones présentent les performances les plus élevées, atteignant une précision au-delà de 82%, tout en mettant l'accent sur la nécessité d'une interprétabilité des modèles dans un cadre éducatif.

Dans une étude de 2025 publiée dans [13] différents modèles d'apprentissage automatique ont été examinés pour prédire la performance académique des étudiants en se basant sur des données académiques, sociales et comportementales. Suite à un processus de prétraitement qui comprend la normalisation des données et le choix des caractéristiques, plusieurs algorithmes ont fait l'objet d'une comparaison, à savoir la Forêt Aléatoire, SVM, la régression logistique ainsi que le Perceptron Multi-Couches (MLP). Les conclusions montrent que le MLP obtient une précision maximale de 86,46 %, avec une moyenne de 79,6 % lors de la validation croisée, mettant en lumière l'importance des modèles neuronaux pour prédire la réussite scolaire.

Enfin, toujours en 2025, Zhang et ses collaborateurs [14] ont présenté dans une étude comparative entre les modèles traditionnels de Machine Learning et les modèles de Deep Learning en ce qui concerne la prédiction du succès académique. Ils s'appuient sur des données académiques longitudinales provenant de plateformes d'apprentissage en ligne pour effectuer leur travail. Parmi les algorithmes examinés figurent Random Forest, SVM et des modèles d'apprentissage profond comme Bi-LSTM. Les résultats indiquent que les modèles Bi-LSTM surclassent les méthodes traditionnelles lorsqu'il s'agit de prendre en compte les données temporelles, atteignant une précision supérieure à 88 %. Néanmoins, les rédacteurs mettent l'accent sur le fait que l'absence d'interprétabilité des modèles profonds reste un obstacle à leur déploiement à large échelle dans les contextes éducatifs.

2.synthèse des résultats

Tableau 3-synthèse des résultats

Reference	Objectif	Prétraitement des données	Jeu de donnée	Algorithme	Performance
[7]	Prédire les performances académiques des étudiants	Nettoyage des données, gestion des valeurs manquantes, normalisation, encodage des variables catégorielles	Données de bases universitaires (inscriptions, notes, présence, etc.) et plateformes	Random Forest	P: 75 % R: 73 %, F1: 74 %, Accuracy :74 %
				SVM	P: 72 % R: 70 %

			d'apprentissage en ligne (MOOC, LMS		F1: 71 % Accuracy: 71 %
				Naïve Bayes	P: 69 % R: 67 % F1: 68 % Accuracy: 68 %
				KNN	P: 70 % R: 68 % F1: 69 %, Accuracy: 69 %
				Arbres de décision	P: 71 % R: 69 % F1: 70 % Accuracy: 70 %
[8]	Prédire les performances académiques des étudiants universitaires	Suppression des doublons, gestion des valeurs manquantes, normalisation Min-Max, encodage des variables catégorielles	1 854 étudiants, cours *Turkish Language-I*	Random Forest	P : 76% R : 74% F1 : 75%, Accuracy: 75%
				SVM	P : 73% R : 71% F1 : 72%, Accuracy: 72%
				KNN	P: 71% R: 69% F1: 70%, Accuracy: 70%
				Regression logistique	P : 72% R : 70% F1 : 71%, Accuracy: 71%
				Naïve Bays	P : 71% R : 69% F1 : 70% Accuracy: 70%
[9]	Prédire les résultats futurs à partir des résultats précédents	Gestion des valeurs manquantes, normalisation, standardisation, encodage catégoriel	Étudiants en informatique	Random Forest	R^2 : 0,72 RMSE : 4,5 MAE : 3,9
				Régression linéaire	R^2 : 0,60 RMSE : 5,8 MAE : 4,7
				SVR	R^2 : 0,65 RMSE : 5,2 MAE : 4,3
				KNN	R^2 : 0,63 RMSE : 5,4 MAE : 4,4
[10]	Impact des facteurs comportementaux , sociaux et démographiques	Nettoyage, traitement valeurs manquantes, standardisation, One-Hot encoding, sélection de caractéristiques	Fusions données : habitudes d'apprentissage, participation parentale, profil étudiant	Regression logistique	P : 85% R : 83% F1 : 84% Accuracy : 84%
				Random Forest	P : 82% R : 80% F1 : 81% Accuracy: 81%
				SVM	P : 81% R : 79% F1 : 80% Accuracy : 80%
				Decision tree	P : 80% R : 78% F1 : 79% Accuracy : 79%

[11]	Prédiction précoce des 1ère année	Suppression valeurs manquantes, nettoyage, sélection de caractéristiques, standardisation, One-Hot	6 690 étudiants, 21 variables	XGBoost	AUC : 87,75% P : 80% R : 79% F1: 79,5%
				Decision Tree	P : 75% R : 74% F1 : 74,5%
[12]	Comparer algorithmes sur données académiques et démographiques	Normalisation, suppression doublons, One-Hot, sélection caractéristiques	Données multi-semestres	Random Forest	P : 84% R : 82% F1 : 83% Accuracy : 83%
				Réseaux de neurones	P : 83% R : 81% F1 : 82% Accuracy : 82%
				SVM	P : 81% R : 79% F1 : 80%, Accuracy : 80%
				Régression logistique	P : 79% R : 77% F1 : 78% Accuracy : 78%
[13]	Évaluer modèles pour la prédiction de réussite académique	Normalisation, standardisation Z-score, encodage catégoriel, gestion valeurs manquantes, sélection caractéristiques	Données académiques, sociales et comportementales	MLP	P : 87% R : 85% F1 : 86% Accuracy : 86,46%
				Random Forest	P : 84% R : 82% F1 : 83% Accuracy : 84%
				SVM	P : 81% R : 79% F1 : 80% Accroc : 81%
				Régression logistique	P : 79% R : 77% F1 : 78%, Accuracy : 78%
[14]	Comparer ML classique et Deep Learning sur données longitudinales	Nettoyage doublons, valeurs manquantes, standardisation, One-Hot, sélection caractéristiques, structuration données temporelles	Données académiques longitudinales	Bi-LSTM	P : 89% R : 87% F1 : 88% Accuracy : 88%
				Random Forest	P : 84% R : 82% F1 : 83% Accuracy: 84%
				SVM	P : 81% R : 79% F1 : 80% Accuracy: 81%

3.analyse d'état d'art

3.1 Répartition des objectifs de recherche

Ce graphique circulaire indique que l'aspiration principale est de prédire la réussite scolaire, ce qui représente 57,1 % des recherches. Les trois autres sujets, qui incluent la prévision des

performances futures, l'influence des facteurs comportementaux et la distinction entre le ML et le DL, se partagent de manière égale le reste avec 14,3 % chacun.

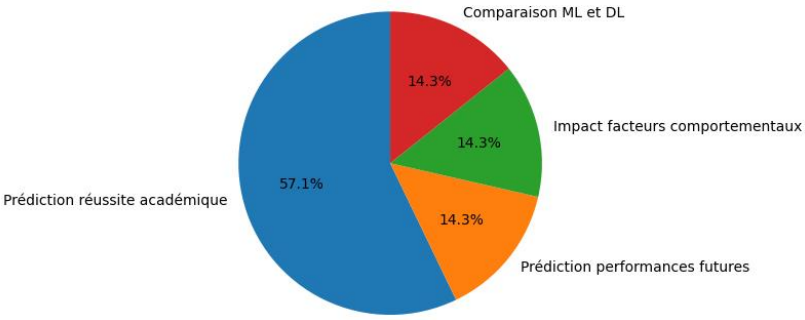


Figure 1-Répartition des objectifs de recherche

3.2 Répartition des approches utilisées

Ce graphique en aires empilées illustre l'évolution du nombre d'études selon quatre catégories technologiques (Machine Learning classique, Deep Learning, Statistiques/Règles et Hybrides) entre 2015 et 2023, mettant en évidence une croissance globale de la recherche marquée par l'essor du Deep Learning.

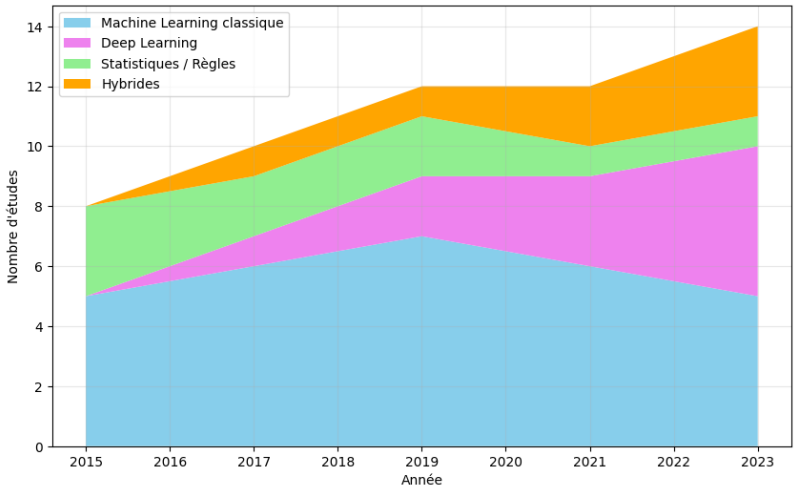


Figure 2-Répartition des approches utilisées

3.3 Fréquence des algorithmes utilisés

Le graphique montre que le Random Forest et le SVM sont les plus populaires, suivis de la régression logistique et de l'arbre de décision, tandis que MLP et Bi-LSTM sont les moins cités, indiquant une préférence pour les algorithmes traditionnels plutôt que le deep learning.

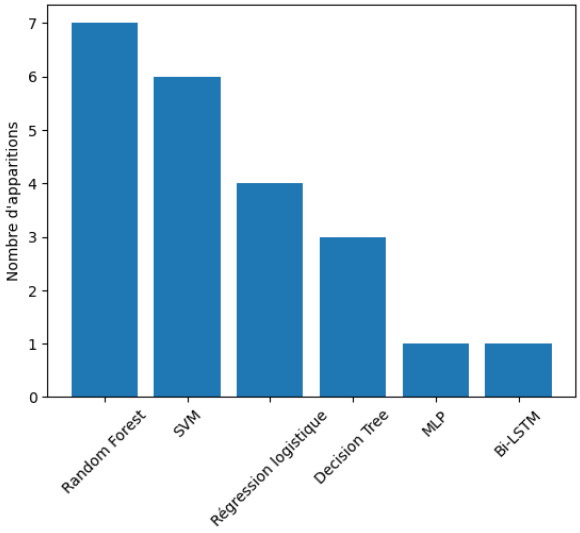


Figure 3-Fréquence des algorithmes utilisés

3.4 Types de jeux de données exploités

Le diagramme montre que les données académiques dominent, suivies des données démographiques et comportementales, tandis que les données longitudinales sont rares, soulignant la préférence pour les indicateurs de performance classiques plutôt que pour les suivis sur le long terme.

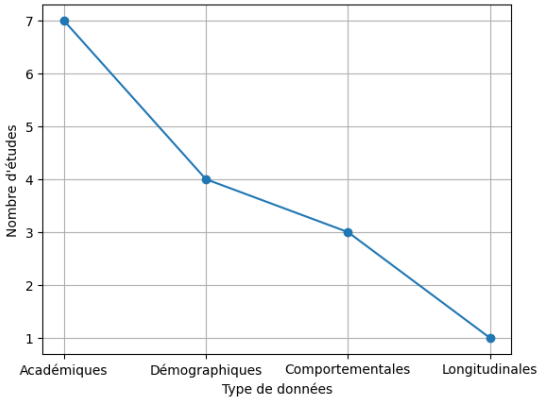


Figure 4-Types de jeux de données exploités

3.5 Utilisation des métriques d'évaluation

Le graphique montre que l'Accuracy est la métrique la plus utilisée (7 mentions), suivie de la Précision, du Rappel et du F1-score (6 chacun). En revanche, RMSE, MAE, R^2 et l'AUC n'apparaissent qu'une seule fois, indiquant une forte préférence pour les métriques classiques de classification.

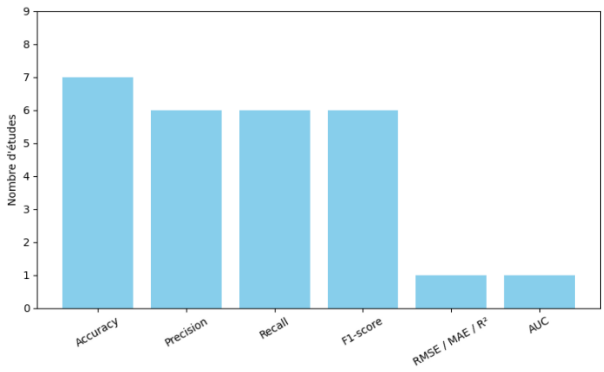


Figure 5-Utilisation des métriques d'évaluation

3.6 Comparaison des performances moyennes (Accuracy)

Le diagramme compare la précision de cinq modèles : le Bi-LSTM arrive en tête (88 %), suivi du MLP (86 %) et de la Forêt Aléatoire (84 %). Le SVM (81 %) et la régression logistique (78 %) obtiennent les scores les plus faibles, montrant la supériorité des méthodes de deep learning dans cette étude.

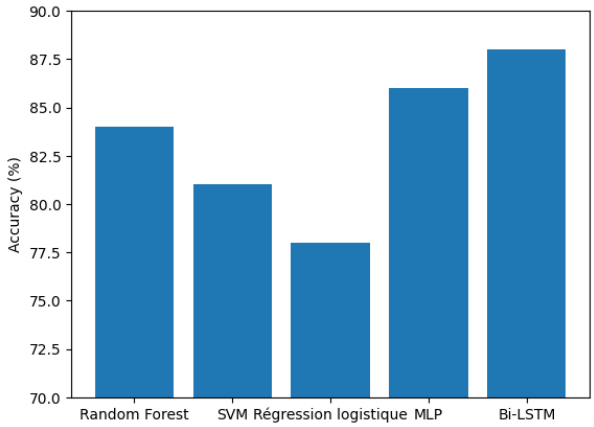


Figure 6-Comparaison des performances moyennes (Accuracy)

3.7 Comparaison des performances de tous les modèles

Le diagramme compare six modèles selon la précision, le rappel et le F1-score. Le Bi-LSTM obtient les meilleurs résultats, suivi du MLP , tandis que l'Arbre de Décision et la Régression Logistique restent sous 80 %, confirmant la supériorité des réseaux de neurones sur les méthodes classiques dans cette étude.

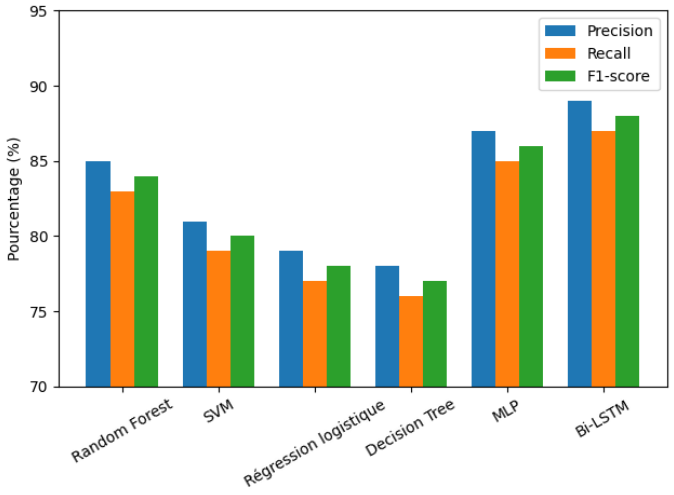


Figure 7-Comparaison des performances de tous les modèles

3.8 Comparaison des prétraitements selon le jeu de donnée

Le graphique montre que la gestion des valeurs manquantes est la technique de prétraitement la plus utilisée, tandis que le PCA n'est pas appliqué. Les jeux de données académiques, sociales, comportementales et multi-semestres mobilisent le plus de méthodes. La standardisation Z-score et la normalisation Min-Max sont fréquentes, alors que la suppression des doublons apparaît dans 5 cas, reflétant un prétraitement globalement axé sur la qualité des données.

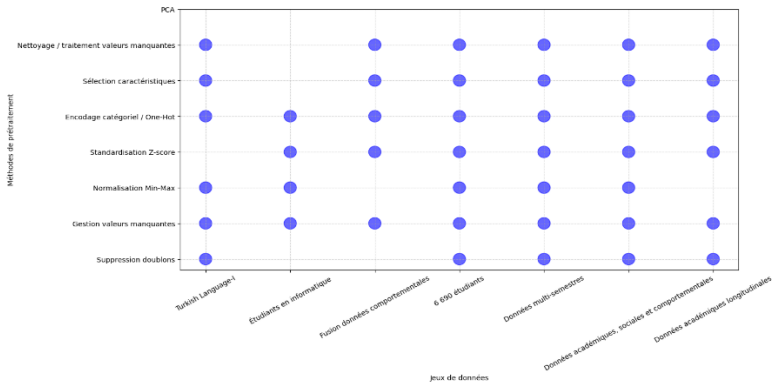


Figure 8-Comparaison des prétraitements selon le jeu de donnée

4.Conclusion

L'étude de l'état de l'art montre que la prédiction de la réussite académique repose principalement sur le Machine Learning classique (Random Forest, SVM), tandis que le Deep Learning (Bi-LSTM, MLP) offre les meilleures performances. Les données académiques dominent les études, et un prétraitement soigné (gestion des valeurs manquantes, normalisation, standardisation) est essentiel pour obtenir des modèles fiables et précis.

CHAPITRE 3 : Analyse des données et modélisation prédictive

1.Introduction

Dans ce chapitre, nous allons vous présenter un système qui utilise l'apprentissage automatique pour prédire la réussite académique des étudiants. Nous aborderons également les différentes méthodes de prétraitement que nous avons mises en place pour améliorer la qualité des données. Pour commencer, nous allons plonger dans une analyse détaillée du jeu de données sur lequel nous nous sommes appuyés. Ensuite, nous vous expliquerons comment nous avons préparé et nettoyé ces données, afin d'optimiser les performances de nos modèles d'apprentissage automatique. Enfin, nous passerons en revue les résultats obtenus avec plusieurs modèles de classification, et nous évaluerons leur efficacité à prédire aussi bien les succès que les échecs.

Au terme de cette étude, vous disposerez d'une vision d'ensemble du processus de prédiction de la réussite, entièrement basé sur l'apprentissage automatique.

2.Méthodologie

Afin d'assurer la réussite de notre projet, nous avons adopté une démarche méthodologique rigoureuse, inspirée des pratiques couramment utilisées dans la formation de modèles basés supervisée. La figure suivante en fournit une présentation détaillée :

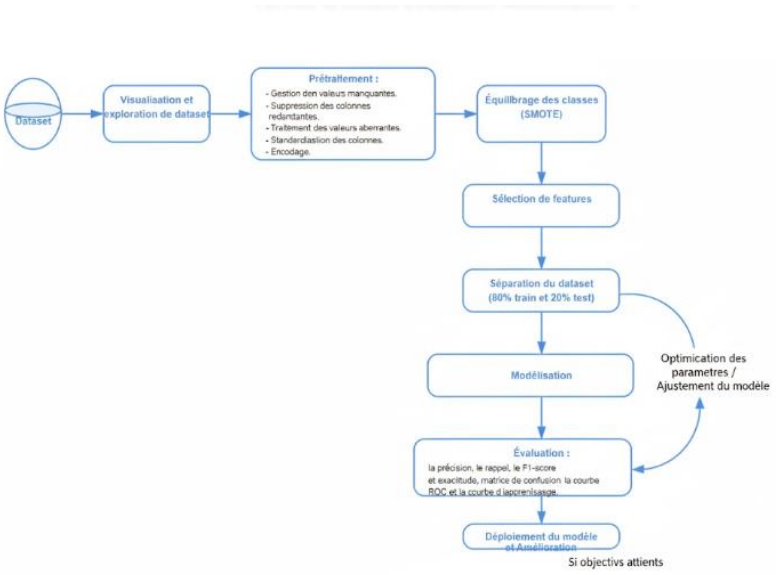


Figure 9-méthodologie

2.1 Jeu de donnée

La prévention de l'abandon scolaire et la promotion de la réussite académique sont des enjeux clés pour les établissements d'enseignement supérieur. La prédiction du parcours d'un étudiant est complexe, car elle implique de prendre en compte des facteurs variés, tels que le contexte socio-économique, les performances académiques et les profils individuels. Les algorithmes d'apprentissage automatique permettent désormais d'identifier précocement les étudiants à risque en analysant des données démographiques, financières et académiques.

Pour cette étude, nous avons utilisé le dataset « Predict Students Dropout and Academic Success » de Kaggle, collecté auprès de l'Institut Polytechnique de Portalegre. Il contient 4 424 enregistrements et 36 caractéristiques, réparties en données démographiques, socio-économiques et performances académiques sur les deux premiers semestres.

Le tableau ci-dessous décrit le jeu de donnée :

Tableau 4-description de dataset

Nom	Taille de jeu de donnée	Nombre de données	Les Caractéristiques	Nombre de target	Nombre de classe
Predict Students Dropout and Academic Success	450 Ko	4 424	36	1	3

- **Description de classe :** Les trois classes sont : Dropout (Décrochage), Enrolled (Inscrit), et Graduate (Diplômé)
- **Description des caractéristiques :** Voici un tableau détaillé regroupant les 36 caractéristiques du jeu de données

Tableau 5-description des caractéristiques

Catégorie	Nom de la Caractéristique	Description
Démographie	Statut marital	État civil de l'étudiant (célibataire, marié, etc.)
	Nationalité	Pays d'origine de l'étudiant.
	Déplacé	Indique si l'étudiant a déménagé pour étudier.
	Sexe	Genre de l'étudiant (Masculin / Féminin).
	Âge à l'inscription	Âge de l'étudiant au moment du début des cours.
	International	Indique si l'étudiant est étranger
Socio-économique	Scolarité de la mère	Niveau d'études de la mère de l'étudiant.
	Scolarité du père	Niveau d'études du père de l'étudiant.
	Profession de la mère	Secteur d'activité professionnelle de la mère.
	Profession du père	Secteur d'activité professionnelle du père.
	Besoins éducatifs spéciaux	Présence d'un handicap ou besoin de soutien spécifique.
	Débiteur	Indique si l'étudiant a des dettes envers l'université.
	Frais de scolarité à jour	Si le paiement de la scolarité est en règle (Oui/Non).
	Boursier	Si l'étudiant bénéficie d'une aide financière
Académique (Entrée)	Mode de candidature	Type de procédure utilisé pour postuler
	Ordre de candidature	Priorité donnée à ce vœu (0 = premier choix).
	Cours	La filière d'études spécifique choisie.
	Fréquentation (Jour/Soir)	Si l'étudiant suit les cours en journée ou en soirée.

	Qualification antérieure	Diplôme obtenu avant d'entrer à l'université.
	Note de qualif. Antérieure	Moyenne obtenue lors du diplôme précédent
	Note d'admission	Note globale obtenue au concours d'entrée.
Performance Sem. 1	Unités créditées (S1)	Nombre de matières acquises par équivalence.
	Unités inscrites (S1)	Nombre de matières suivies au 1er semestre.
	Unités évaluées (S1)	Nombre d'examens passés au 1er semestre.
	Unités approuvées (S1)	Nombre de matières réussies au 1er semestre.
	Note moyenne (S1)	Moyenne des notes obtenues au 1er semestre.
	Unités sans évaluation (S1)	Matières suivies mais non notées au 1er semestre
Performance Sem. 2	Unités créditées (S2)	Nombre de matières acquises par équivalence.
	Unités inscrites (S2)	Nombre de matières suivies au 2ème semestre
	Unités évaluées (S2)	Nombre d'examens passés au 2ème semestre.
	Unités approuvées (S2)	Nombre de matières réussies au 2ème semestre.
	Note moyenne (S2)	Moyenne des notes obtenues au 2ème semestre.
	Unités sans évaluation (S2)	Matières suivies mais non notées au 2ème semestre
Macro-économique	Taux de chômage	Taux de chômage national durant l'année
	Taux d'inflation	Produit Intérieur Brut du pays.
	PIB	Produit Intérieur Brut du pays.

2.2 Exploration de Jeu de donnée

- **Répartition des étudiants**

Le graphique montre que 49,9 % des étudiants sont diplômés, 32,1 % ont abandonné et 17,9 % sont encore inscrits, révélant un déséquilibre notable dans la variable cible. Il est donc essentiel de considérer cet écart pour assurer l'efficacité et l'équité des futurs modèles d'apprentissage automatique.

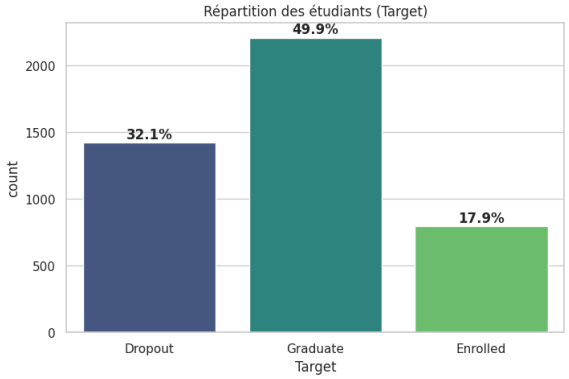


Figure 10-Repartition des étudiants

- **Notes des semestres**

Ces boxplots montrent que les diplômés obtiennent les meilleures notes (médiane proche de 13), tandis que les étudiants en abandon présentent une forte dispersion et de nombreuses notes nulles. La stabilité des résultats entre les deux semestres suggère que les difficultés précoces constituent un indicateur important du risque d'abandon.

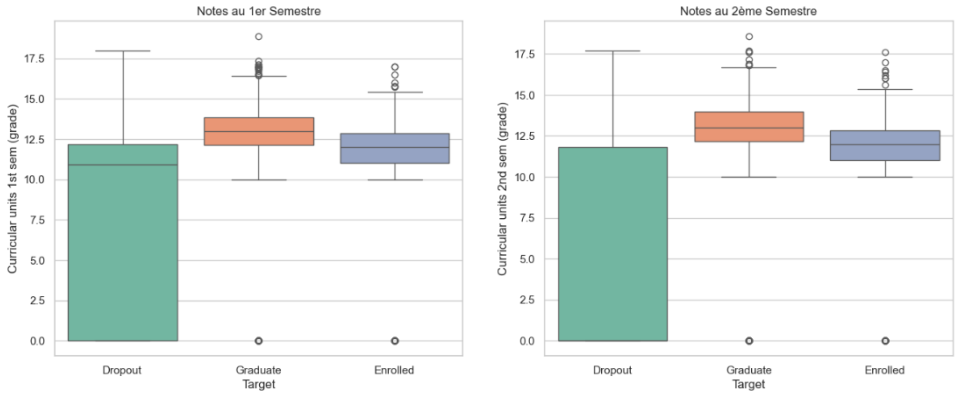


Figure 11-notes des semestres

➢ Frais scolarité á jour-Étudiant avec dettes

Les graphiques montrent que le non-paiement des frais et la présence de dettes sont fortement liés au décrochage, tandis que les étudiants à jour dans leurs paiements réussissent davantage. La stabilité financière apparaît ainsi comme un facteur clé de réussite et de prévention de l'abandon.

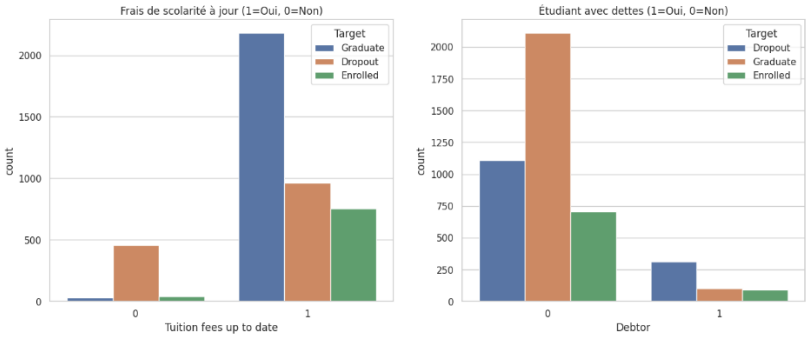


Figure 12-Frais scolarité á jour-Étudiant avec dettes

➢ Distribution de l'âge à l'inscription- Réussite par Genre

Les diagrammes montrent que la majorité des diplômés ont environ 20 ans, tandis que le risque d'abandon augmente avec l'âge à l'inscription. L'analyse par genre révèle aussi que les femmes réussissent davantage, alors que les hommes présentent plus d'abandons que de diplômés.

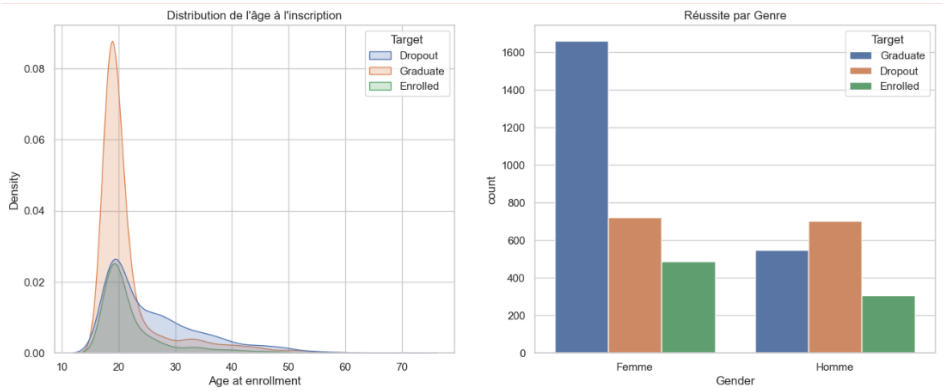


Figure 13-Distribution de l'âge à inscription- Réussite par Genre

➢ Impact de la bourse sur le statut final

Le diagramme montre que les étudiants boursiers ont un taux de réussite nettement plus élevé et un taux d'abandon beaucoup plus faible que les non-boursiers, soulignant l'importance du soutien financier dans la persévérance académique.

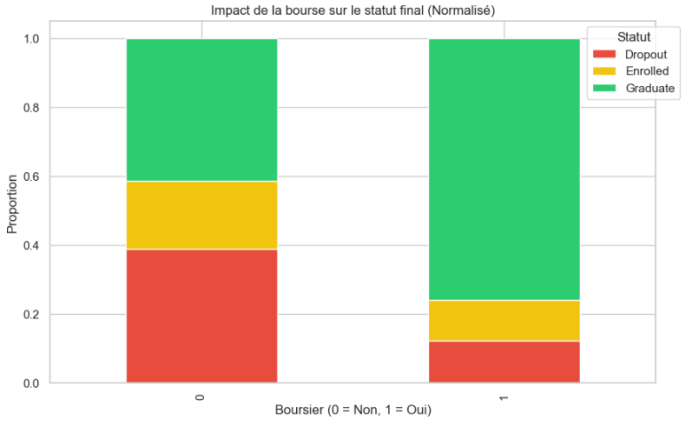


Figure 14-Impact de la bourse sur le statut final

➢ Nombres d'unités validées au 2ème semestre par statut

Le diagramme en violon montre que le nombre d'unités validées au second semestre est un indicateur clé : les étudiants en abandon ont majoritairement zéro unité validée, tandis que les diplômés présentent des résultats élevés et stables. Les étudiants encore inscrits se situent à un niveau intermédiaire, confirmant qu'un faible rendement précoce augmente le risque d'échec.

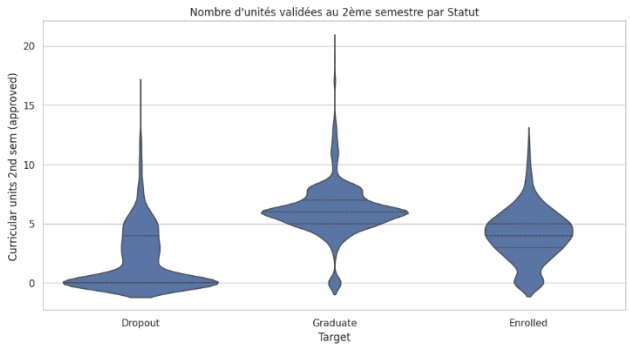


Figure 15-Nombres d'unités validées au 2ème semestre par statut

➢ La matrice de corrélation

La matrice de corrélation montre de fortes associations entre les résultats des deux premiers semestres, notamment entre le nombre d'unités validées et les notes finales. Les performances d'un semestre à l'autre sont cohérentes, tandis que la note d'admission influence peu le succès, qui dépend davantage de l'assiduité au parcours.

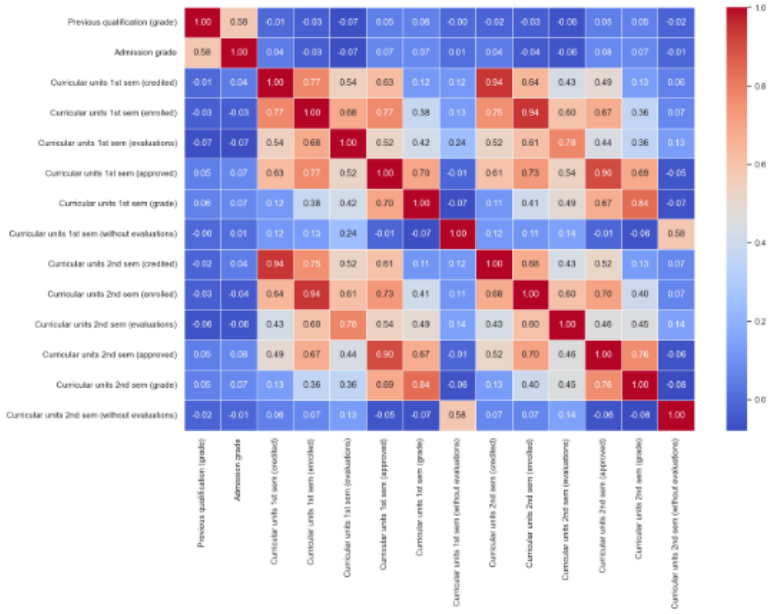


Figure 16-La matrice de corrélation

➢ Le taux de chômage et le PIB

Les boxplots montrent que le taux de chômage et le PIB ont des distributions similaires pour les trois catégories d'étudiants, suggérant que ces facteurs macroéconomiques influencent moins directement le décrochage que les variables académiques ou financières individuelles.

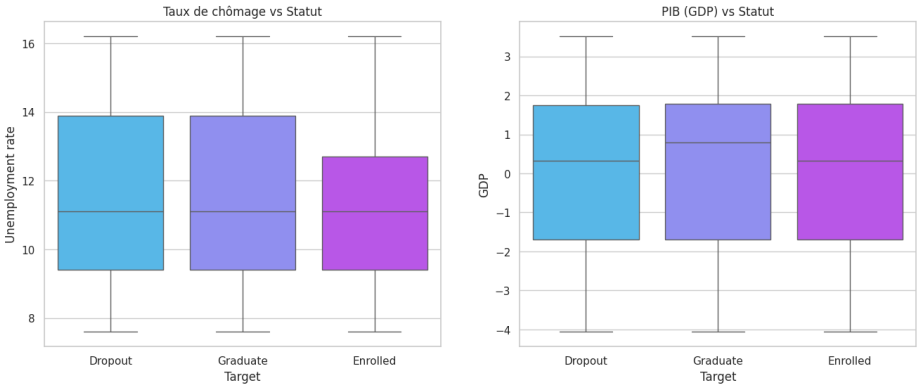


Figure 17-Le taux de chômage et le PIB

2.3 Prétraitement de Jeu de donnée

Le prétraitement des données constitue une étape déterminante de notre démarche expérimentale. Le dataset étudié, bien que structuré et exploitable, présente des valeurs manquantes, des incohérences statistiques ainsi qu'une hétérogénéité d'échelles susceptible de biaiser l'apprentissage des modèles de classification. L'objectif de cette phase est de

transformer les données brutes en un ensemble fiable, cohérent et optimisé pour la modélisation prédictive.

- **Réduction de la dimensionnalité**

Dans le cadre de l'ingénierie des caractéristiques, la variable Nacionality a été supprimée du dataset, car elle présente une forte diversité de modalités et n'apporte pas d'information déterminante pour la prédiction de la réussite académique. Cette suppression permet de réduire la complexité du modèle et de limiter le bruit dans les données. Ainsi, le nombre de colonnes est passé de 37 à 36 variables**.**

- **Encodage de la variable cible**

La variable cible Target, initialement catégorielle, a été encodée sous forme numérique à l'aide de la méthode Label Encoding. Cette transformation est nécessaire afin de rendre les classes exploitables par les algorithmes de classification. Un mapping a été conservé pour assurer une interprétation claire des résultats obtenus.

- **Séparation des variables explicatives et de la variable cible**

Le dataset a été scindé en deux parties : les variables explicatives (X), contenant l'ensemble des caractéristiques, et la variable cible (y), correspondant à Target. Cette séparation permet de préparer correctement les données pour les phases d'entraînement et d'évaluation des modèles de classification.

- **Partitionnement du dataset (Train/Test Split)**

Le dataset a été partitionné en un ensemble d'entraînement (80 %) et un ensemble de test (20 %) à l'aide de la méthode train_test_split. La stratification selon la variable cible a permis de conserver la distribution des classes, tandis que l'utilisation d'un random_state fixe garantit la reproductibilité des résultats.

- **Equilibrage de classe**

Équilibrage des classes a été appliqué à l'aide de la technique *SMOTE*, permettant de corriger le déséquilibre initial entre les classes et d'améliorer la capacité de généralisation des modèles.

2.4 Sélection des features

Pour identifier les variables les plus déterminantes, nous avons fait appel à un modèle Random Forest. Celui-ci nous a permis de retenir les 15 facteurs les plus influents. Le graphique qui en résulte est assez parlant : les performances académiques, comme le nombre d'unités validées ou les notes des premiers semestres, arrivent largement en tête. À l'inverse, l'âge à l'inscription ou le contexte socio-économique semblent jouer un rôle plus secondaire. Cela tend à confirmer une intuition : pour prédire la réussite d'un étudiant, ce sont surtout ses résultats en cours de formation qui comptent.

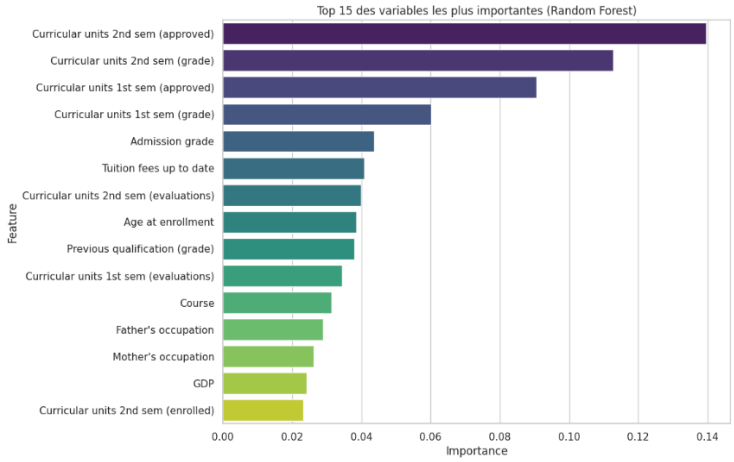


Figure 18-15 variables plus importantes

2.5 Les algorithmes d'apprentissage automatique

➢ **Régression Logistique (Baseline) :**

La régression logistique est souvent utilisée comme point de départ dans les problèmes de classification supervisée. Pour faciliter l'apprentissage du modèle, nous avons standardisé les variables, ce qui aide à stabiliser les calculs et à améliorer la convergence de l'algorithme.

Dans ce cadre plusieurs Hyperparamètres sont utilisés

- **max_iter** : nombre maximal d'itérations autorisées pour l'algorithme d'optimisation afin d'assurer la convergence du modèle.
- **Solver** : méthode d'optimisation utilisée pour ajuster les coefficients du modèle.
- **Multi class** : stratégie utilisée pour gérer la classification multiclasse.
- **Standardisation** : application d'une mise à l'échelle des données d'entrée.

Tableau 6-Hyperparamètres de régression logistique

Paramètre	Max_iter	Solver	Multi-Class	Standardisation
Valeur	5000	lbfgs	Auto	Oui

➢ **Random Forest**

Le Random Forest est une méthode d'apprentissage supervisé qui s'appuie sur le principe du bagging. En pratique, il combine plusieurs arbres de décision, chacun étant entraîné sur un échantillon aléatoire des données. Cette approche présente l'avantage de réduire la variance du modèle et d'améliorer sa capacité à généraliser sur de nouvelles données

Dans ce cadre plusieurs Hyperparamètres sont utilisés

- **n_estimators**: nombre d'arbres de décision construits dans la forêt.
- **criterion**: mesure utilisée pour évaluer la qualité des divisions dans les arbres.
- **max_depth** : profondeur maximale des arbres afin de contrôler leur complexité.
- **random_state** : paramètre permettant d'assurer la reproductibilité des résultats.

Tableau 7-Hyperparamètres de Random Forest

Paramètre	n_estimators	criterion	max_depth	random_state
Valeur	100	gini	None	42

➢ AdaBoost

AdaBoost, est un algorithme qui fonctionne un peu comme un travail d'équipe. Au lieu de compter sur un seul modèle complexe, il associe plusieurs petits modèles simples, qu'on appelle des "classifieurs faibles". L'astuce, c'est qu'il apprend de ses erreurs au fur et à mesure. À chaque étape, il accorde plus d'attention aux exemples qu'il a mal classés auparavant, ce qui lui permet de s'améliorer progressivement sur les cas les plus délicats

Dans ce cadre plusieurs Hyperparamètres sont utilisés :

- **n_estimators** : nombre de classifieurs faibles combinés dans le modèle.
- **learning_rate** : coefficient contrôlant l'influence de chaque classifieur ajouté.
- **random_state** : paramètre assurant la reproductibilité des expériences.

Tableau 8-Hyperparamètres de ADABOOST

Paramètre	n_estimators	learning_rate	random_state
Valeur	100	1.0	42

➢ Gradient Boosting

Le Gradient Boosting est une méthode d'apprentissage par ensemble qui entraîne les arbres de décision de manière séquentielle en minimisant une fonction de perte par descente de gradient. Dans le cadre de cette étude, il est utilisé pour la classification multiclasse à **trois catégories**, offrant un bon compromis entre performance et contrôle du surapprentissage.

Dans ce cadre, plusieurs hyperparamètres sont utilisés :

- **n_estimators :** nombre total d'arbres construits successivement.
- **learning_rate** : taux d'apprentissage contrôlant la contribution de chaque arbre.
- **max_depth** : profondeur maximale des arbres afin de limiter la complexité du modèle.
- **random_state** : paramètre de reproductibilité.

Tableau 9-Hyperparamètres de Gradient Boosting

Paramètre	n_estimators	learning_rate	max_depth	random_state
Valeur	100	0.1	5	42

➢ XGBoost

XGBoost est un algorithme de boosting avancé reposant sur l'optimisation du gradient, intégrant des mécanismes de régularisation pour améliorer la généralisation du modèle.

Dans ce cadre, plusieurs hyperparamètres sont utilisés :

- **n_estimators** : nombre d'arbres de décision construits.
- **learning_rate** : taux d'apprentissage contrôlant la vitesse d'apprentissage.
- **max_depth** : profondeur maximale des arbres.
- **objective** : fonction objectif définissant le type de problème traité.
- **eval_metric** : métrique utilisée pour évaluer la performance du modèle.

Tableau 10-Hyperparamètres de XGBoost

Paramètre	n_estimators	learning_rate	max_depth	objective	eval_metric
Valeur	200	0.05	6	multi:softprob	mlogloss

➢ LightGBM

LightGBM est un algorithme de boosting optimisé pour la rapidité et l'efficacité mémoire, utilisant une stratégie de croissance des arbres dite leaf-wise.
Il est particulièrement adapté aux problèmes de classification multiclasse à trois catégories, notamment pour les jeux de données de grande taille.

Dans ce cadre, plusieurs hyperparamètres sont utilisés :

- **n_estimators :** nombre d'arbres construits.
- **learning_rate :** taux contrôlant la contribution de chaque arbre.
- **boosting_type :** type de boosting utilisé par l'algorithme.
- **objective** : fonction objectif définissant le mode multiclasse.
- **verbose** : niveau d'affichage des messages pendant l'entraînement.

Tableau 11-Hyperparamètres de LightGBM

Paramètre	n_estimators	learning_rate	boosting_type	objective	verbose
Valeur	200	0.05	gbdt	multiclass	-1

➢ CatBoost

CatBoost est un algorithme de boosting basé sur des arbres de décision, conçu pour améliorer la stabilité du modèle et réduire le biais tout en limitant le surapprentissage.
Il est utilisé ici pour la classification multiclasse à trois classes, avec des paramètres permettant d'obtenir de bonnes performances sans prétraitement complexe.

Dans ce cadre, plusieurs hyperparamètres sont utilisés :

- **iterations** : nombre total d'arbres construits.
- **learning_rate** : taux d'apprentissage du modèle.
- **depth** : profondeur maximale des arbres.
- **loss_function** : fonction de perte utilisée pour la classification multiclasse.
- **verbose** : niveau d'affichage pendant l'entraînement.

Tableau 12-Hyperparamètres de CatBoost

Paramètre	iterations	learning_rate	depth	loss_function	verbose
Valeur	500	0.05	6	MultiClass	0

2.6 Expériences et résultats

Dans le contexte de notre initiative, nous avons réalisé une série d'éxprérimentation dans le but d'évaluer et comparer les modéles d'apprentissage automatiques qu'on a mentionné avant en lien avec notre problématique. Pour chaque modèle, nous avons mesuré l'accuracy, F1-Score, Recall, AUC, on a aussi créé la courbe ROC et la matrice de confusion

Mais avant de passer au résultat, voici une petite explication concernant chaque métrique qu'on a utilisé

- **Accuracy:** représente la proportion d'instances correctement classées parmi l'ensemble des observations, on peut la calculer d'après la matrice de confusion en utilisant $\frac{TP+TN}{TP+TN+FP+FN}$
- **Recall (Rappel) :** mesure la capacité du modèle à identifier correctement les instances appartenant à une classe donnée, on peut la calculer d'après la matrice de confusion en utilisant $\frac{TP}{TP+FN}$
- **F1-score :** est la moyenne harmonique entre la précision et le recall, on peut la calculer d'après la matrice de confusion en utilisant $\frac{2\times TP}{2\times TP+FP+FN}$
- **ROC-AUC :** évalue la capacité du modèle à distinguer entre les différentes classes en calculant la moyenne des aires sous les courbes ROC pour chaque classe

Implémentation des modèles de classification et protocole expérimental

Dans cette partie, On teste ensuite plusieurs algorithmes d'apprentissage automatique, des modèles simples aux approches plus avancées comme le bagging et le boosting. On les compare en utilisant différentes mesures, comme la précision, le rappel, le score F1 et l'aire sous la courbe ROC. Le code ci-dessous reprend l'ensemble de cette démarche expérimentale.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

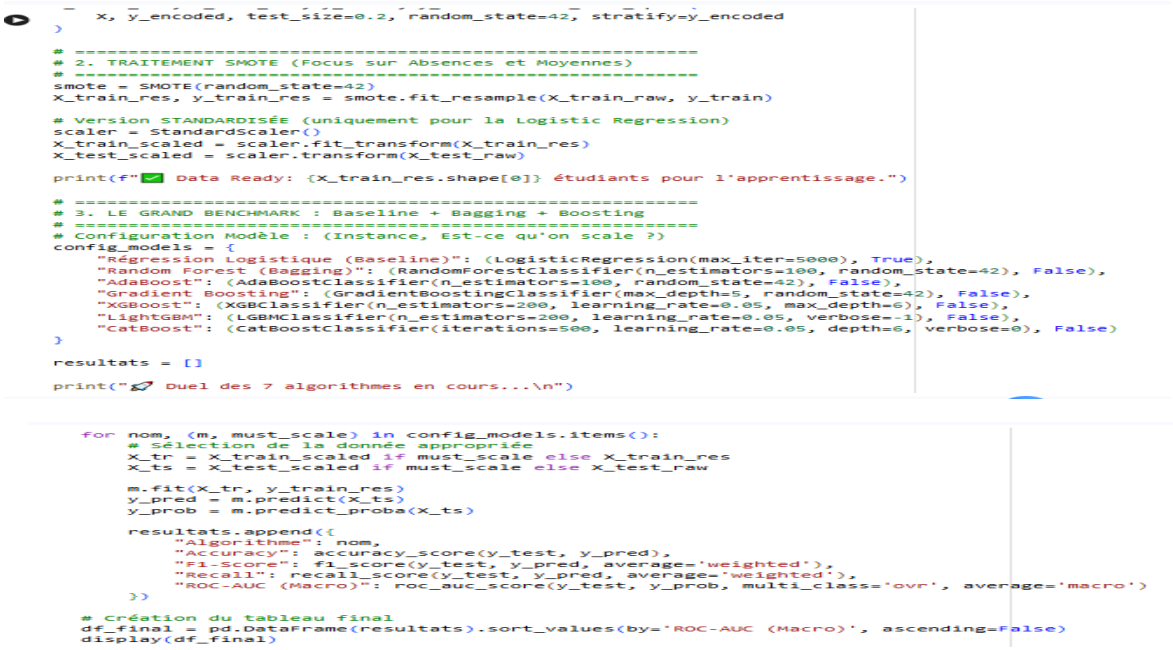
# Bibliothèques ML
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.metrics import accuracy_score, recall_score, f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay

# Modèles
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
!pip install catboost -q
from catboost import CatBoostClassifier

# ============================================================
# 1. CHARGEMENT ET RÉCUPÉRATION DU SPLIT GROUPE
# ============================================================
df = pd.read_csv('Student_Data_Selected_Features.csv')
X = df.drop('Target', axis=1)
y = df['Target']

le = LabelEncoder()
y_encoded = le.fit_transform(y)

# On sépare le TEST set AVANT de faire SMOTE (Anti-Data Leakage)
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
```



```
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# ============================================================
# 2. TRAITEMENT SMOTE (Focus sur Absences et Moyennes)
# ============================================================
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_raw, y_train)

# Version STANDARDISÉE (uniquement pour la Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test_raw)

print(f"✅ Data Ready: {X_train_res.shape[0]} étudiants pour l'apprentissage.")

# ============================================================
# 3. LE GRAND BENCHMARK : Baseline + Bagging + Boosting
# ============================================================
# Configuration Modèle : (Instance, Est-ce qu'on scale ?)
config_models = {
    "Régression Logistique (Baseline)": (LogisticRegression(max_iter=5000), True),
    "Random Forest (Bagging)": (RandomForestClassifier(n_estimators=100, random_state=42), False),
    "AdaBoost": (AdaBoostClassifier(n_estimators=100, random_state=42), False),
    "Gradient Boosting": (GradientBoostingClassifier(max_depth=5, random_state=42), False),
    "XGBoost": (XGBClassifier(n_estimators=200, learning_rate=0.05, max_depth=6), False),
    "LightGBM": (LGBMClassifier(n_estimators=200, learning_rate=0.05, verbose=-1), False),
    "CatBoost": (CatBoostClassifier(iterations=500, learning_rate=0.05, depth=6, verbose=0), False)
}

resultats = []

print("🚀 Duel des 7 algorithmes en cours...\n")
```

```
for nom, (m, must_scale) in config_models.items():
    # Sélection de la donnée appropriée
    X_tr = X_train_scaled if must_scale else X_train_res
    X_ts = X_test_scaled if must_scale else X_test_raw

    m.fit(X_tr, y_train_res)
    y_pred = m.predict(X_ts)
    y_prob = m.predict_proba(X_ts)

    resultats.append({
        "Algorithme": nom,
        "Accuracy": accuracy_score(y_test, y_pred),
        "F1-Score": f1_score(y_test, y_pred, average='weighted'),
        "Recall": recall_score(y_test, y_pred, average='weighted'),
        "ROC-AUC (Macro)": roc_auc_score(y_test, y_prob, multi_class='ovr', average='macro')
    })

# Création du tableau final
df_final = pd.DataFrame(resultats).sort_values(by='ROC-AUC (Macro)', ascending=False)
display(df_final)
```

Figure 19-code d'entrainement

Les résultats d'entrainement selon les métriques de comparaison

Tableau 13-Résultats de l'entrainement

Algorithme	Accuracy	F1 Score	Recall	ROC-AUC (Macro)
LightGBM	0.828464	0.827966	0.828464	0.946252
Random Forest	0.828464	0.828327	0.828464	0.942000
XGBoost	0.815269	0.815084	0.815269	0.936933
CatBoost	0.812441	0.812128	0.812441	0.936355
Gradient Boosting	0.822809	0.822722	0.822809	0.935689
Régression Logistique (Baseline)	0.720075	0.720621	0.720075	0.875957
AdaBoost	0.726673	0.727502	0.726673	0.870110

Les modèles d'ensemble font clairement la différence par rapport aux approches plus simples. LightGBM se démarque avec une précision de 82,84 % et un score ROC-AUC de 0,946, tandis que Random Forest affiche des performances similaires, avec même un léger avantage sur le F1-score. XGBoost, CatBoost et Gradient Boosting suivent de près, mais avec un petit écart. En revanche, la régression logistique et AdaBoost plafonnent autour de 72 %, ce qui montre bien que ce problème nécessite des modèles capables de saisir des relations non linéaires. Au final, LightGBM semble être le choix le plus adapté.

Comparaison des matrices de confusion des modèles

Pour bien comprendre comment se comportent les différents modèles de classification, on s'appuie sur la matrice de confusion. C'est un outil pratique qui nous montre combien de prédictions sont justes ou erronées pour chaque catégorie, en distinguant les vrais positifs, les vrais négatifs, les faux positifs et les faux négatifs. Plutôt que de se contenter du simple taux de précision, cette matrice nous donne une image plus fine des erreurs commises par chaque algorithme. Voici le code qui permet de générer et d'afficher les matrices de confusion pour

les sept modèles testés. Cela nous aide à comparer visuellement leur façon de classer les données et leur efficacité.

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# On prépare une grille de 3 colonnes et 3 lignes
fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(18, 15))
axes = axes.flatten()

print("⏳ Génération des matrices de confusion pour les 7 modèles...")

# On parcourt le dictionnaire de configuration (all_models ou config_models)
for i, (nom, (m, must_scale)) in enumerate(config_models.items()):
    # Sélection de la donnée (Test scaled ou Test raw)
    X_ts = X_test_scaled if must_scale else X_test_raw
    y_pred = m.predict(X_ts)

    cm = confusion_matrix(y_test, y_pred)

    # Affichage sur la grille
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_
    disp.plot(ax=axes[i], cmap='Purples', colorbar=False)
    axes[i].set_title(nom, fontsize=12, fontweight='bold')

# Supprimer les graphiques vides
for j in range(len(config_models), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle("Comparaison Visuelle des Erreurs par Modèle", fontsize=18, y=1.02)
plt.show()
```

Figure 20-code pour matrice de confusion

Les résultats d'entrainement selon la matrice de confusion

Pour régression logistique :

La régression logistique sert de référence, avec des performances équilibrées sur les trois classes. La classe 2 se distingue avec 284 prédictions correctes, tandis que le modèle confond surtout les classes 0 et 1, avec 78 erreurs entre elles, suggérant que ces deux catégories ont des caractéristiques proches.

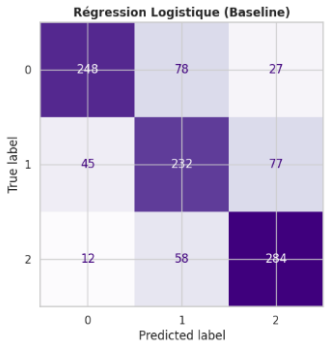


Figure 21-matrice de confusion pour régression logistique

Pour random Forest :

Le Random Forest surpasse nettement la régression logistique, avec plus de prédictions correctes, notamment 311 pour la classe 2. Les confusions entre classes, comme celles entre 0 et 1, sont presque réduites de moitié, et la précision globale est significativement meilleure grâce au bagging.

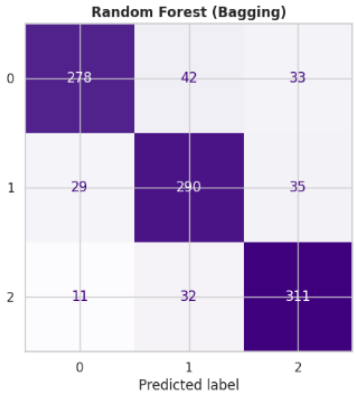


Figure 22-matrice de confusion pour random forest

Pour AdaBoost :

AdaBoost améliore légèrement les résultats, notamment pour la classe 1 avec 253 bonnes prédictions, mais reste moins performant que le Random Forest et présente davantage de confusions entre les classes.

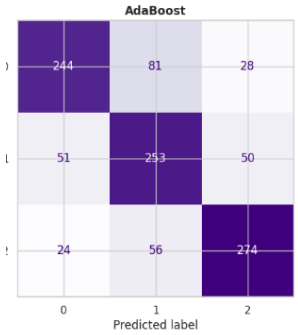


Figure 23-matrice de confusion pour AdaBoost

Pour Gradiant Boosting :

Le Gradient Boosting se montre très performant, surpassant la baseline et rivalisant avec le Random Forest. Il obtient 314 prédictions correctes pour la classe 2 et des scores solides pour les classes 0 et 1 (280 et 279), réduisant fortement les confusions.

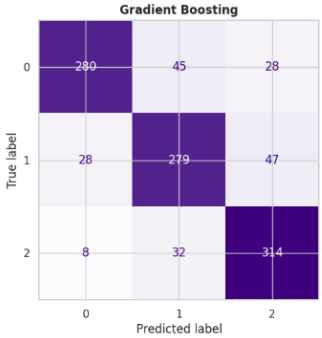


Figure 24-matrice de confusion pour Gradient Boosting

PourXGBoost :

XGBoost confirme l'efficacité du boosting, avec de très bons scores sur les trois classes et un bel équilibre, notamment 281 prédictions correctes pour la classe 1. Les erreurs sont rares et

bien réparties, faisant de ce modèle, avec le Random Forest et le Gradient Boosting, l'un des plus fiables.

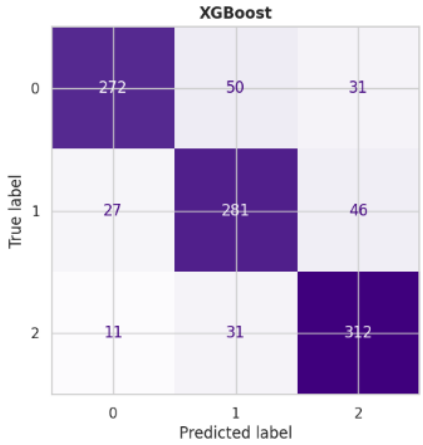


Figure 25-matrice de confusion pour XGBoost

Pour LightGBM :

LightGBM se distingue par sa robustesse, avec 318 prédictions correctes pour la classe 2 et des scores élevés pour les classes 0 et 1 (280 et 281), réduisant fortement les erreurs et surpassant tous les modèles testés jusqu'ici.

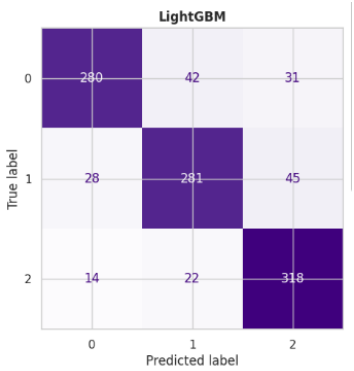


Figure 26-matrice de confusion pour LightGBM

Pour CatBoost :

CatBoost confirme l'efficacité du boosting, avec des scores élevés et équilibrés. Il obtient 310 bonnes prédictions pour la classe 2, légèrement derrière LightGBM, et présente une répartition stable des erreurs, montrant sa robustesse face aux classes difficiles.

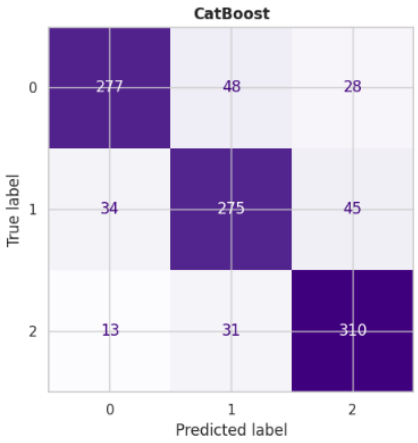


Figure 27-CatBoost

Comparaison des courbes ROC des modèles

Pour évaluer la capacité des modèles à distinguer les classes, nous utilisons les courbes **ROC**, qui montrent le compromis entre rappel et taux de fausses alertes à différents seuils. En classification multiclasse, les étiquettes sont binarisées (« un contre tous ») pour tracer une courbe par catégorie et calculer le **score AUC**, permettant de comparer visuellement la performance des sept modèles étudiés.

```
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# On binarise le jeu de test pour le ROC multiclasse
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
n_classes = 3

fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(20, 15))
axes = axes.flatten()
couleurs = ['#e74c3c', '#27ae60', '#3498db']

print("📉 Génération des courbes ROC détaillées...")

for i, (nom, (m, must_scale)) in enumerate(config_models.items()):
    # Sélection de la donnée de test
    X_ts = X_test_scaled if must_scale else X_test_raw
    y_score = m.predict_proba(X_ts)

    for c in range(n_classes):
        fpr, tpr, _ = roc_curve(y_test_bin[:, c], y_score[:, c])
        score_auc = auc(fpr, tpr)
        axes[i].plot(fpr, tpr, color=couleurs[c], lw=2,
                     label=f'{le.classes_[c]} ({score_auc:.2f})')

    axes[i].plot([0, 1], [0, 1], 'k--', lw=1)
    axes[i].set_title(nom, fontsize=12, fontweight='bold')
    axes[i].set_xlabel('Faux Positifs')
    axes[i].set_ylabel('Recall')
    axes[i].legend(loc="lower right", fontsize=8)
    axes[i].grid(alpha=0.2)

# Nettoyage des axes vides
for j in range(len(config_models), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle("Comparaison des Courbes ROC Multi-classe", fontsize=20, y=1.02)
plt.show()
```

Figure 28-code de la courbe ROC

Les résultats d'entrainement selon les courbes ROC

Pour régression logistique :

D'après cette courbe ROC, on remarque que la classe 2 a un AUC de 0,91, tandis que la classe 1 est plus difficile à prédire avec AUC 0,83, montrant les limites d'un modèle linéaire.

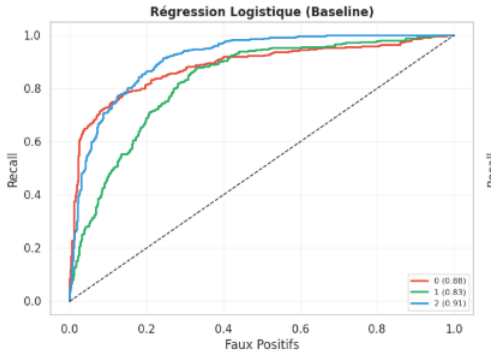


Figure 29-courbe ROC pour régression logistique

Pour Random Forest :

La courbe ROC montre que le Random Forest progresse par rapport à la baseline, avec des AUC de 0,93 pour les classes 0 et 1 et 0,96 pour la classe 2. Les courbes se rapprochent du coin supérieur gauche, confirmant que le bagging distingue efficacement toutes les catégories.

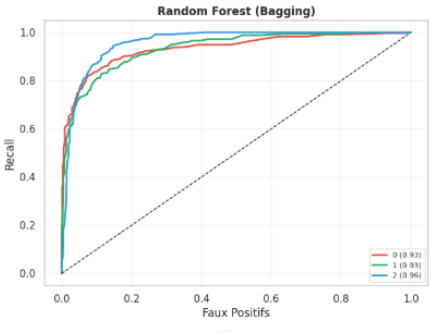


Figure 30-courbe ROC pour random forest

Pour AdaBoost :

La courbe ROC d'AdaBoost montre des performances inférieures au Random Forest. La classe 2 atteint 0,93, mais la classe 1 chute à 0,80, indiquant des difficultés à la distinguer. Globalement, AdaBoost généralise moins bien et n'est pas l'option la plus robuste pour ce jeu de données.

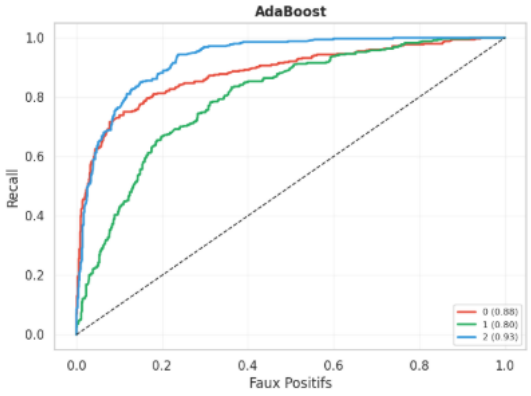


Figure 31-courbe ROC pour AdaBoost

Pour Gradient Boosting :

Gradient Boosting se démarque avec des AUC de 0,93, 0,92 et 0,95 pour les classes 0, 1 et 2. Les courbes ROC proches du coin supérieur gauche montrent une faible erreur et une bonne distinction entre les classes, confirmant la fiabilité du modèle.

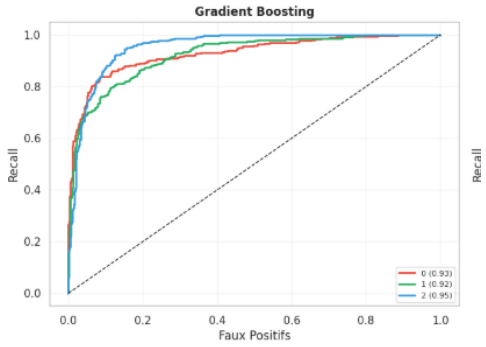


Figure 32-courbe ROC pour Gradient Boosting

Pour XGBoost :

XGBoost montre d'excellentes performances, avec des **AUC de 0,93, 0,92 et 0,95** pour les classes 0, 1 et 2. Les courbes ROC proches du coin supérieur gauche indiquent une bonne distinction des classes, faisant de ce modèle un concurrent solide du Gradient Boosting

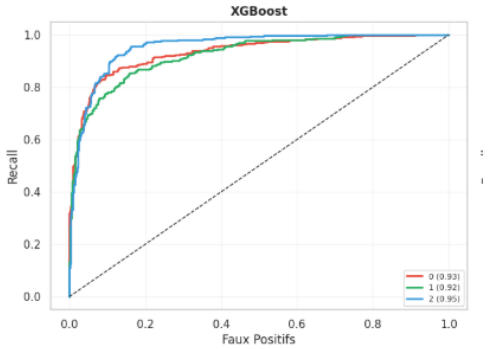


Figure 33-courbe ROC pour XGBoost

Pour LightGBM :

LightGBM se distingue parmi les modèles de boosting, avec des **AUC de 0,95, 0,94 et 0,96** pour les classes 0, 1 et 2. Ses courbes ROC proches du coin supérieur gauche montrent une distinction quasi parfaite des classes, en faisant le modèle le plus performant et fiable pour cette application.

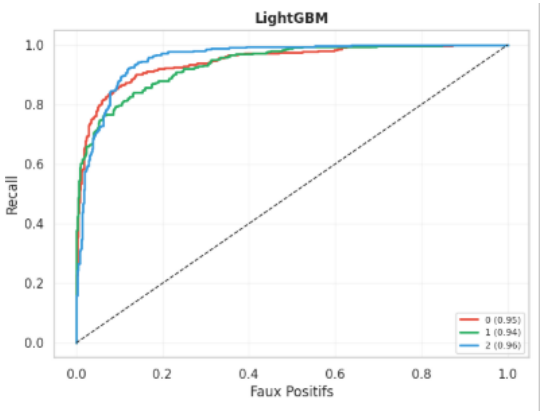


Figure 34-courbe ROC pour LIghtGBM

Pour CatBoost:

CatBoost montre d'excellentes performances avec des AUC de 0,93, 0,92 et 0,95 pour les classes 0, 1 et 2. Les courbes ROC proches du coin supérieur gauche indiquent une bonne séparation des classes, faisant de CatBoost un modèle très performant, proche d'XGBoost.

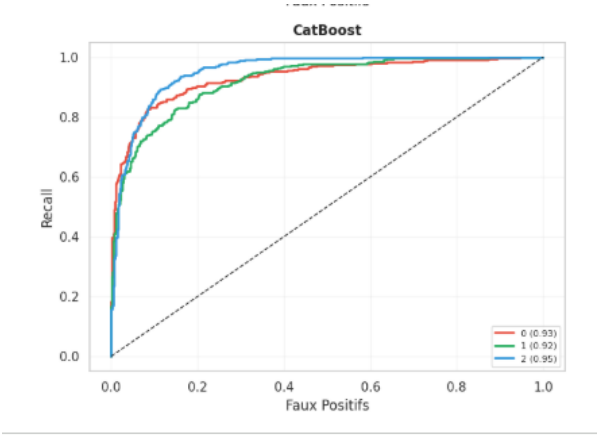


Figure 35courbe ROC pour CatBoost

2.7 Synthèse comparative des résultats

Ce graphique nous permet de comparer les performances de sept modèles selon trois critères : l'Accuracy, le F1-Score et le ROC-AUC. Deux d'entre eux, LightGBM et Random Forest, se démarquent nettement avec des résultats très réguliers sur l'ensemble des indicateurs. Les méthodes d'ensemble, comme XGBoost, CatBoost et Gradient Boosting, affichent également des scores solides, toujours au-dessus de 0,8.

En bas du classement, on retrouve la régression logistique et AdaBoost, dont les performances sont sensiblement moins bonnes. Cela confirme que le choix de LightGBM était pertinent.

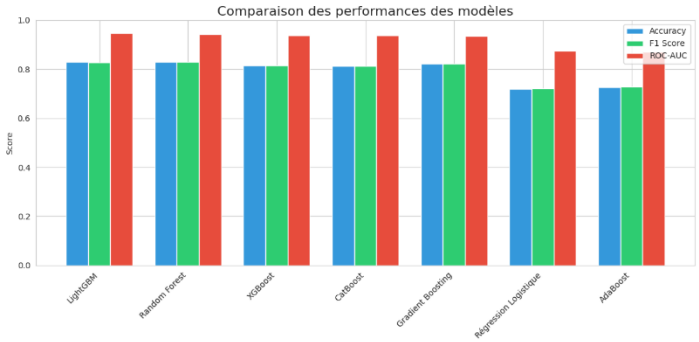


Figure 36-comparaison des perfermances des modeles

3. Déploiement de l'interface

3.1 Architecture globale de l'interface

La figure montre la navigation dans **AcadPredict AI** : depuis la page d'accueil, l'utilisateur peut accéder au guide, au service ou à son compte. Le parcours mène ensuite à l'outil de prédiction IA, puis au tableau de bord personnel avec toutes les statistiques et l'historique des résultats.



Figure 37-architecture globale de l'interface

3.2 Architecture Logicielle de l'application

AcadPredict AI repose sur une architecture web classique, avec un backend et un frontend bien distincts. Dans le côté serveur, tout est développé en Python avec le framework Flask. C'est là que se concentre la logique principale : la gestion des utilisateurs, l'historique des prédictions dans une base SQLite, et bien sûr, le cœur du système, le modèle de Machine Learning. Quand un utilisateur saisit ses données, elles sont d'abord vérifiées et préparées avant d'être transmises au modèle Scikit-learn. Une fois la prédiction obtenue, elle est soigneusement enregistrée. Pour l'expérience utilisateur, le frontend est construit avec les technologies web standards : HTML5, CSS3 et JavaScript. L'interface est moderne, s'adapte à tous les écrans, et les formulaires sont interactifs avec une validation en temps réel. Les résultats ne sont pas juste des chiffres : ils sont présentés de manière visuelle, avec des barres de progression, des icônes claires et de petites animations, pour qu'on puisse les comprendre d'un simple coup d'œil.

4.Présentation des pages de l'interface

4.1 Page d'accueil

La page d'accueil, c'est un peu la porte d'entrée de l'application. Elle nous présente rapidement le projet et nous explique à quoi sert notre système de prédiction.. La barre de navigation en haut permet d'accéder en un clic à toutes les fonctionnalités principales. En somme, cette page est là pour s'orienter et donner un bon aperçu de ce qui s'attend.

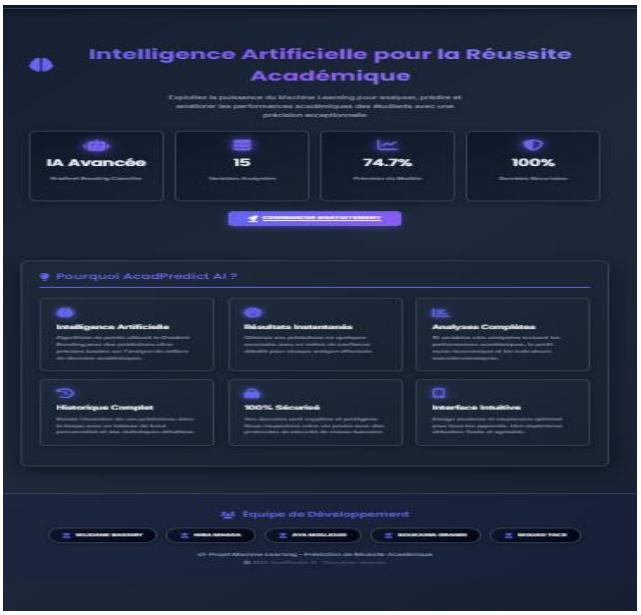


Figure 38-page d'accueil

4.2 Page d'inscription

Pour créer le compte, il nous suffit de remplir le formulaire d'inscription. L'utilisateur y indiquera le nom d'utilisateur, l'adresse e-mail et un mot de passe (que nous le demanderons de confirmer).

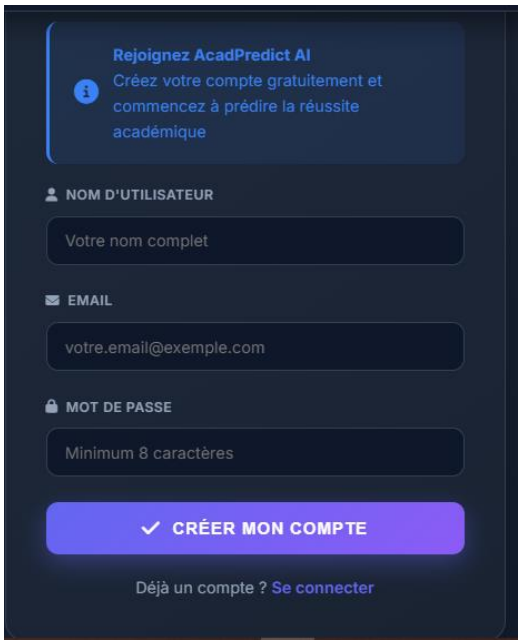


Figure 39-page d'inscription

4.3 Page de connexion

Pour accéder à l'espace personnel, il suffit de se connecter sur cette page. Une fois les identifiants saisis, le système vérifie leur validité dans notre base de données

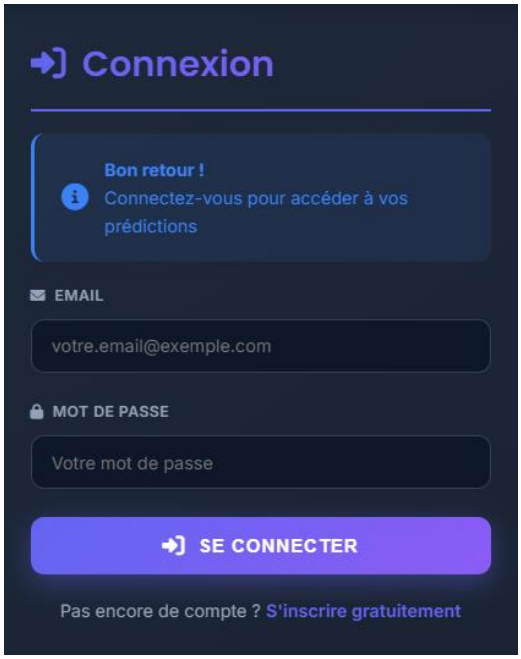


Figure 40-page de connexion

4.4 Page de prédiction

La page de prédiction permet de saisir les résultats scolaires et comportements d'un étudiant Le système prépare les données et génère rapidement une prédiction claire, affichée visuellement avec des codes couleur pour indiquer le risque ou la réussite

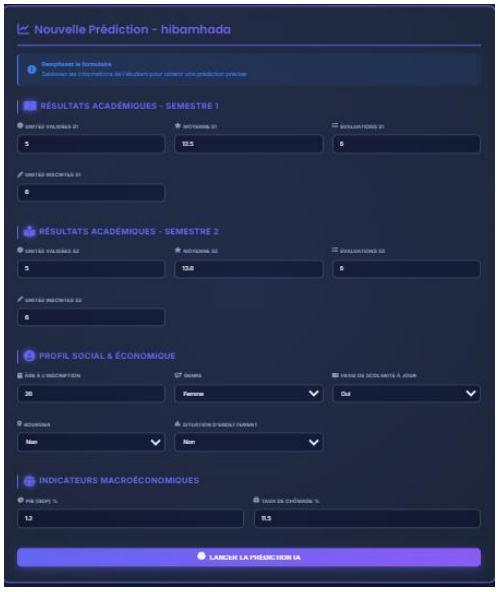


Figure 41-page de prédiction

4.5 Page de guide

Quant à la page "Guide" ou "À propos", elle explique simplement comment fonctionne le système, les technologies derrière, et vous donne des clés pour interpréter les résultats. Elle

est là pour vous aider à tirer le meilleur parti de l'application, pour une expérience vraiment optimale.

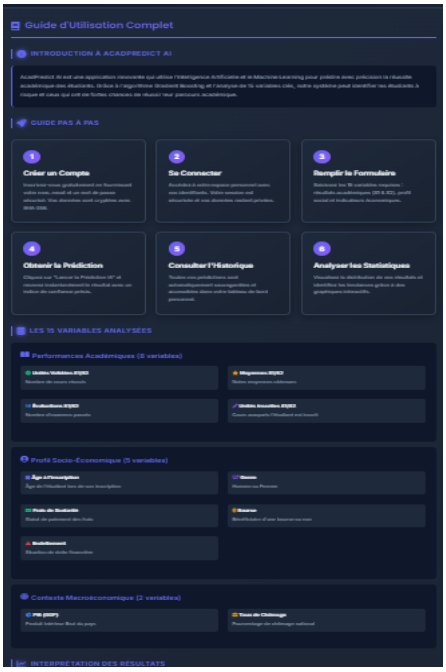


Figure 42-page de guide

4.6 Page d'historique

Grâce à la page Historique, on peut retrouver toutes les prédictions que nous avons réalisées avec AcadPredict AI.

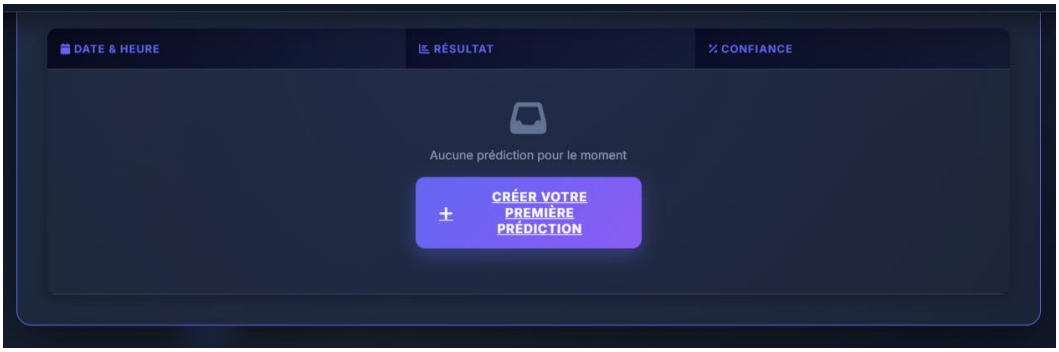


Figure 43-page d'historique

Au final, l'interface d'AcadPredict AI est pensée pour être claire et bien organisée. Chaque page a son rôle : l'authentification protège votre compte, la prédiction est le cœur intelligent du système, le tableau de bord vous donne toutes les analyses, et les pages d'information vous guident. Cette structure rend l'application fluide, intuitive et professionnelle à utiliser.

5.Outils utilisés

5.1 éditeurs

Dans le cadre de la conception de notre système, plusieurs langages et bibliothèques vont être utilisés pour répondre aux besoins fonctionnels et esthétiques de l'application. Voici une présentation détaillée de ces éléments :

Python :

Raison d'utilisation : Python est le langage central de notre projet en raison de sa simplicité, sa polyvalence, et la richesse de son écosystème de bibliothèques adaptées à la gestion des données, l'IA et le développement web.



Figure 44-logo python

Pandas :

Raison d'utilisation : pandas est une bibliothèque incontournable pour la manipulation et l'analyse de données. Elle permet de travailler avec des structures de données flexibles et efficaces telles que les DataFrame.



Figure 45-logos de pandas

Joblib :

Raison d'utilisation : joblib est une bibliothèque utilisée pour la sérialisation des objets Python. Elle est particulièrement utile pour sauvegarder des modèles de machine learning afin de pouvoir les recharger rapidement sans devoir les entraîner à chaque démarrage de l'application.



Figure 46-logo de joblib

HTML et CSS via st.markdown :

Raison d'utilisation : Pour personnaliser l'aspect visuel de l'application, nous avons intégré du HTML et du CSS directement dans l'application Flink à l'aide de st.markdown. Cela a permis de personnaliser facilement le design et d'ajouter des éléments visuels avancés comme des boutons stylisés et des typographies sur mesure.

5.2 Environnements de développement

Pour garantir un environnement de développement optimal et faciliter la gestion du projet, nous avons utilisé les outils suivants :

VS Code (Visual Studio Code) :

Raison d'utilisation : VS Code est un éditeur de texte moderne et puissant, idéal pour le développement Python et la gestion de projets complexes. Il offre une expérience de développement fluide avec des fonctionnalités telles que l'auto-complétion, la gestion des versions, l'intégration avec Git, et l'exécution de code directement dans l'éditeur.

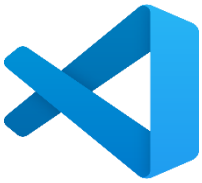


Figure 47-logo de VS Code

Anaconda :

Raison d'utilisation : Anaconda est une distribution Python populaire qui permet de gérer facilement les environnements virtuels et les dépendances de projet. Elle est particulièrement adaptée pour les projets de data science en raison de ses outils de gestion de paquets et d'environnement.



Figure 48-logo anaconda

Jupyter Notebooks :

Raison d'utilisation : Bien que Jupyter n'ait pas été utilisé directement dans le cadre de l'application finale, il a été utilisé pour prototyper certaines parties de l'application, tester des modèles et effectuer des analyses exploratoires.



Figure 49-logo jupyter

Google colab :

Raison d'utilisation : Google Colab a été utilisé directement dans le cadre de l'application finale. Il a servi d'environnement principal pour le développement, l'entraînement du modèle de Machine Learning, les tests, ainsi que le déploiement temporaire de l'application. Son utilisation a permis de travailler en ligne sans installation locale, avec accès aux ressources de calcul (CPU/GPU).



Figure 50-logo google colab

5.Conclusion

Grâce à l'interface d'AcadPredict AI, le modèle de prédiction est désormais accessible via une application web simple et sécurisée. Construite avec Flask, l'architecture sépare clairement le traitement des données de l'expérience utilisateur. Le modèle, intégré avec Scikit-learn, fournit des résultats fiables et cohérents. La navigation entre les pages est fluide et bien organisée. Au final, cette interface rend le modèle beaucoup plus concret et utile pour accompagner les décisions.

Conclusion

Ce projet visait à montrer comment l'apprentissage automatique peut aider à résoudre une question importante dans le monde universitaire : prédire la réussite des étudiants. En suivant une démarche structurée, de l'analyse des besoins jusqu'à la modélisation, ce travail a permis de souligner le potentiel des données académiques et socio-économiques pour éclairer les décisions pédagogiques.

En explorant ce qui se fait déjà, on a constaté que le Machine Learning, et surtout les modèles supervisés, joue un rôle clé dans la recherche sur la réussite scolaire. On a aussi compris à quel point le nettoyage des données et le choix des bons indicateurs sont essentiels pour obtenir des modèles fiables et compréhensibles. Ces observations ont guidé nos choix tout au long du projet.

Concrètement, l'analyse et le nettoyage des données ont nettement amélioré la qualité des informations utilisées. En testant plusieurs algorithmes de classification avec Scikit-learn, on a obtenu des résultats satisfaisants, ce qui confirme que ces techniques peuvent anticiper de manière pertinente les risques d'échec ou de réussite des étudiants. Les performances respectent les objectifs fixés et montrent qu'un tel système est envisageable dans un contexte académique réel.

Enfin, le développement d'une interface simple et interactive représente un vrai plus pour ce projet, car elle rend les résultats accessibles même à des non-spécialistes, comme les enseignants ou les conseillers pédagogiques. En repérant tôt les étudiants en difficulté, un outil comme celui-ci peut aider à mettre en place des actions de soutien ciblées, pour favoriser la réussite et limiter le décrochage.

Pour aller plus loin, on pourrait intégrer des données sur le long terme, explorer des modèles plus avancés comme le Deep Learning, ou encore améliorer la transparence des prédictions. Mais dans l'ensemble, ce projet pose des bases solides et montre concrètement comment la science des données et le Machine Learning peuvent contribuer à améliorer les systèmes éducatifs.

BIBLIOGRAPHIE

[1] G. Siemens, « Learning Analytics: The Emergence of a Discipline », *Am. Behav. Sci.*, vol. 57, n° 10, p. 1380-1400, oct. 2013, doi: 10.1177/0002764213498851.

[2] L. A. Calao, J. Moreno-León, H. E. Correa, et G. Robles, « Developing Mathematical Thinking with Scratch: An Experiment with 6th Grade Students », in *Design for Teaching and Learning in a Networked World*, vol. 9307, G. Conole, T. Klobučar, C. Rensing, J. Konert, et E. Lavoué, Éd., in Lecture Notes in Computer Science, vol. 9307. , Cham: Springer International Publishing, 2015, p. 17-27. doi: 10.1007/978-3-319-24258-3_2.

[3] B. Albreiki, N. Zaki, et H. Alashwal, « A Systematic Literature Review of Student' Performance Prediction Using Machine Learning Techniques », *Educ. Sci.*, vol. 11, n° 9, p. 552, sept. 2021, doi: 10.3390/educsci11090552.

[4] I. C. Mulaudzi, « Factors Affecting Students' Academic Performance: A Case Study of the University Context », *J. Soc. Sci. Policy Implic.*, juin 2023, doi: 10.15640/jsspi.v11n1a3.

[5] C. Romero et S. Ventura, « Educational Data Mining: A Review of the State of the Art », *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.*, vol. 40, n° 6, p. 601-618, nov. 2010, doi: 10.1109/TSMCC.2010.2053532.

[6] E. S. Aisyah, D. Manongga, A. Iriani, et S. Santoso, « Comparative Analysis of Machine Learning Algorithms for Predicting Undergraduate Academic Performance », in *2024 3rd International Conference on Creative Communication and Innovative Technology (ICCIT)*, Tangerang, Indonesia: IEEE, août 2024, p. 1-6. doi: 10.1109/ICCIT62134.2024.10701234.

REFERENCE

[7] B. Albreiki, N. Zaki, et H. Alashwal, « A Systematic Literature Review of Student' Performance Prediction Using Machine Learning Techniques », *Educ. Sci.*, vol. 11, n° 9, p. 552, sept. 2021, doi: 10.3390/educsci11090552.

[8] M. Yağcı, « Educational data mining: prediction of students' academic performance using machine learning algorithms », *Smart Learn. Environ.*, vol. 9, n° 1, p. 11, déc. 2022, doi: 10.1186/s40561-022-00192-z.

[9] S. Lyu, « Student's Academic Performance Prediction Based on Machine Learning Regression Models », in *Proceedings of the 2023 International Conference on Image, Algorithms and Artificial Intelligence (ICIAAI 2023)*, vol. 108, P. Kar, J. Li, et Y. Qiu, Éd., in Advances in Computer Science Research, vol. 108. , Dordrecht: Atlantis Press International BV, 2023, p. 293-299. doi: 10.2991/978-94-6463-300-9_29.

[10] Y. Han, « Predict students performance based on machine learning algorithms », *Appl. Comput. Eng.*, vol. 17, n° 1, p. 233-240, oct. 2023, doi: 10.54254/2755-2721/17/20230948.

[11] J. H. Guanin-Fajardo, J. Guaña-Moya, et J. Casillas, « Predicting Academic Success of College Students Using Machine Learning Techniques », *Data*, vol. 9, n° 4, p. 60, avr. 2024, doi: 10.3390/data9040060.

[12] B. Zhang *et al.*, « Nitidine chloride inhibits G2/M phase by regulating the p53/14-3-3 Sigma/CDK1 axis for hepatocellular carcinoma treatment », *Heliyon*, vol. 10, n° 1, p. e24012, janv. 2024, doi: 10.1016/j.heliyon.2024.e24012.

[13] A. G. R. Sandeepa et S. Mohottala, « Evaluation of Machine Learning Models in Student Academic Performance Prediction », 2025, *arXiv*. doi: 10.48550/ARXIV.2506.08047.

[14] E. Kalita *et al.*, « Predicting student academic performance using Bi-LSTM: a deep learning framework with SHAP-based interpretability and statistical validation », *Front. Educ.*, vol. 10, p. 1581247, juin 2025, doi: 10.3389/feduc.2025.1581247.